

CS336 Assignment 1 (基础): 从零构建 Transformer 语言模型

Building a Transformer LM

Version 26.0.3 · Spring 2026

Stanford CS336 Staff

非官方中文完整翻译

翻译说明。本文依据用户提供的 Stanford CS336 Assignment 1 原始 PDF 完整翻译。章节结构、公式编号、代码、题目名称、评分与提交要求均尽量与原文保持一致；不提供任何作业答案。为保证术语精确，首次出现的重要术语通常保留英文或缩写，例如“标记 (token)”“未归一化分数 (logits)”“旋转位置嵌入 (RoPE)”。库名、API、文件名、命令行与变量名不翻译。

目录

关键技术语对照	3
1 作业概览	4
2 字节对编码 (BPE) 分词器	6
2.1 Unicode 标准	6
2.2 Unicode 编码	7
2.3 子词分词	8
2.4 BPE 分词器训练	8
2.5 BPE 分词器训练实验	10
2.6 BPE 分词器：编码与解码	12
2.6.1 编码文本	12
2.6.2 解码文本	13
2.7 分词器实验	14
3 Transformer 语言模型架构	15
3.1 Transformer LM	15
3.2 补充说明：Batching、Einsum 与高效计算	16
3.2.1 数学记号与内存顺序	18
3.3 基础构件：Linear 与 Embedding 模块	19
3.3.1 参数初始化	19

3.3.2	Linear 模块	19
3.3.3	Embedding 模块	20
3.4	Pre-Norm Transformer Block	21
3.4.1	均方根层归一化 (RMSNorm)	21
3.4.2	逐位置前馈网络	22
3.4.3	相对位置嵌入: RoPE	23
3.4.4	缩放点积注意力	24
3.4.5	因果多头自注意力	25
3.5	完整 Transformer LM	26
4	训练 Transformer 语言模型	28
4.1	交叉熵损失	28
4.2	SGD 优化器	29
4.2.1	在 PyTorch 中实现 SGD	29
4.3	AdamW	31
4.4	学习率调度	33
4.5	梯度裁剪	33
5	训练循环	34
5.1	数据加载器	34
5.2	检查点 (Checkpointing)	35
5.3	训练循环	36
6	生成文本	36
7	实验	38
7.1	如何运行实验与提交结果	38
7.2	TinyStories	38
7.2.1	超参数调节	38
7.2.2	组合全部组件	39
7.2.3	调试模型架构的技巧	39
7.3	消融实验与架构修改	41
7.4	在 OpenWebText 上运行	43
7.5	自行修改模型 + Leaderboard	43
	参考文献	44

关键术语对照

下表给出本文采用的核心术语译法。对容易造成歧义的词，正文会保留英文原词或缩写。

英文术语	本文译法
Language Model (LM)	语言模型
Token / Token ID	标记 (token) / 标记 ID (token ID)
Tokenizer	分词器
Vocabulary (vocab)	词表
Byte-level Byte-Pair Encoding (BPE)	字节级字节对编码 (BPE)
Pre-tokenization / pre-token	预分词 / 预标记 (pre-token)
Merge	合并 (BPE merge)
Embedding	嵌入
Logits	未归一化分数 (logits)
Residual connection / residual stream	残差连接 / 残差流
Pre-norm / post-norm	前置归一化 / 后置归一化
Root Mean Square Layer Normalization (RMSNorm)	均方根层归一化 (RMSNorm)
Position-wise Feed-Forward Network (FFN)	逐位置前馈网络
Gated Linear Unit (GLU)	门控线性单元 (GLU)
Rotary Position Embedding (RoPE)	旋转位置嵌入 (RoPE)
Scaled Dot-Product Attention	缩放点积注意力
Causal mask	因果掩码
Multi-Head Self-Attention (MHA)	多头自注意力 (MHA)
Cross-entropy / Negative log-likelihood	交叉熵 / 负对数似然
Perplexity	困惑度
Optimizer	优化器
Weight decay	权重衰减
Learning-rate schedule	学习率调度
Warm-up	预热 (warm-up)
Cosine annealing	余弦退火
Gradient clipping	梯度裁剪
Checkpoint	检查点
Memory mapping (mmap)	内存映射
Decoding / generation	解码 / 生成
Temperature scaling	温度缩放
Nucleus sampling / top- p sampling	核采样 / top- p 采样
Ablation	消融实验
FLOPs	浮点运算次数 (FLOPs)

Model	FLOPs	Utilization	模型 FLOPs 利用率 (MFU)
-------	-------	-------------	--------------------

1 作业概览

在本次作业中，你将从零构建训练一个标准 Transformer 语言模型 (Language Model, LM) 所需的全部组件，并实际训练若干模型。

你将实现什么

1. 字节对编码 (Byte-Pair Encoding, BPE) 分词器 (第 2 节);
2. Transformer 语言模型 (LM) (第 3 节);
3. 交叉熵损失函数与 AdamW 优化器 (第 4 节);
4. 训练循环，并支持序列化以及加载模型状态和优化器状态 (第 5 节)。

你将运行什么

1. 在 TinyStories 数据集上训练 BPE 分词器;
2. 使用训练好的分词器对数据集编码，将文本转换为整数 ID 序列;
3. 在 TinyStories 数据集上训练 Transformer LM;
4. 使用训练好的 Transformer LM 进行样本生成，并用困惑度 (perplexity) 进行评估;
5. 在 OpenWebText 上训练模型，并将取得的困惑度提交到排行榜。

允许使用的内容

我们希望你从零构建每一个组件。尤其是，除下列内容外，你不得使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义：

- `torch.nn.Parameter`;
- `torch.nn` 中的容器类，例如 `Module`、`ModuleList`、`Sequential` 等；¹
- `torch.optim.Optimizer` 基类。

你可以使用 PyTorch 中的其他定义。如果你想使用某个函数或类，但不确定它是否被允许，可以在课程 Slack 中询问。拿不准时，可以问自己：使用它是否破坏了这次作业“从零实现 (from scratch)”的精神？

关于 AI 工具的说明

AI 已经能够完全自主地解决作业中的许多部分，这会让学生更难真正投入课程材料并从中学学习。

¹完整列表见 <https://pytorch.org/docs/stable/nn.html#containers>。

课程允许使用 AI 工具回答高层概念问题，或提供底层编程文档，例如函数签名和库 API；但不允许使用 AI 工具实现作业的任何部分。这同时包括代码代理（例如 Cursor Agents、Codex、Claude Code）以及 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。当使用 AI agent 时，应确保它遵循课程提供的 `AGENTS.md`；如果使用聊天机器人，也应一并提供所使用的提示词（prompt）。

课程强烈建议你在完成作业时关闭 IDE 中的 AI 自动补全功能（非 AI 的普通函数名补全当然没有问题）。此前学生反馈，关闭 AI 自动补全后更容易深入理解课程材料。完整 AI Policy 请参阅课程所指向的政策文档。

代码结构

作业代码和本说明文档位于：

<https://github.com/stanford-cs336/assignment1-basics>

请使用 Git 克隆仓库；若后续有更新，课程会通知你，此时可以通过 `git pull` 获取最新版。

1. `cs336_basics/*`：你将在这里编写自己的代码。该目录没有预先实现的核心代码，因此你可以从零组织自己的实现。
2. `adapters.py`：测试框架要求你的代码暴露一组特定功能。对于每项功能，例如缩放点积注意力，你只需要在对应 adapter 中调用自己的实现，例如填写 `run_scaled_dot_product_attention`。注意：`adapters.py` 的改动不应包含实质算法逻辑，它只是胶水代码（glue code）。
3. `test_*.py`：这里包含你必须通过的测试，例如 `test_scaled_dot_product_attention`。这些测试会调用 `adapters.py` 中定义的接口。不要修改测试文件。

提交方式

运行 `make_submission.sh` 生成提交用 ZIP。如果你有不希望打包进去的大型数据文件或 checkpoint，请确保把它们加入脚本的排除列表。

你需要向 Gradescope 提交：

- `writeup.pdf`：回答所有书面题，请排版后提交；
- `code.zip`：你编写的全部代码。

如需提交排行榜，向以下仓库提交 PR：

<https://github.com/stanford-cs336/assignment1-basics-leaderboard>

具体说明见该排行榜仓库的 `README.md`。

数据集获取

本作业使用两个预处理后的数据集：TinyStories [R. Eldan et al., 2023] 与 OpenWebText [A. Gokaslan et al., 2019]。它们都以单个大型纯文本文件的形式提供。

如果你正式修读课程，请按照 `compute guide` 的说明下载；如果你自行跟学，则可以使用作业仓库 `README.md` 中的命令下载。

低资源提示：说明

在课程的各次作业说明中，我们会给出在 GPU 资源较少、甚至没有 GPU 时的建议。例如，缩小数据集或模型规模，或者解释如何在 Mac 集成 GPU 或 CPU 上运行训练代码。即使你是能使用课程服务器的 Stanford 学生，这些提示也有助于更快迭代并节省时间，因此建议阅读。

低资源提示：在 Apple Silicon 或 CPU 上完成 Assignment 1

使用教师参考实现，在配备 36 GB 内存的 Apple M4 Max 上，可以在不到 5 分钟内使用 Metal GPU (MPS) 训练出能够生成较流畅文本的语言模型；使用 CPU 则约需 30 分钟。即使你暂时不熟悉这些硬件术语，也只需知道：如果你的笔记本较新、实现正确且效率合理，你完全可以训练出一个能较流畅生成简单儿童故事的小型 LM。后文会说明在 CPU 或 MPS 上需要做哪些调整。

2 字节对编码 (BPE) 分词器

本作业的第一部分将训练并实现一个**字节级 BPE 分词器** [R. Sennrich et al., 2016; C. Wang et al., 2019]。具体来说，我们会把任意 Unicode 字符串表示成字节序列，并在这个字节序列上训练 BPE。随后，再用该分词器把文本（字符串）编码成标记（token，即整数序列），供语言模型使用。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它把字符映射到整数**码点 (code point)**。截至 Unicode 17.0 (2025 年 9 月发布)，该标准在 172 种文字系统中定义了 159,801 个字符。例如，字符“s”的码点是 115，通常记为 U+0073，其中 U+ 是约定前缀，0073 是 115 的十六进制表示；字符“牛”的码点是 29275。

在 Python 中，`ord()` 可以把单个 Unicode 字符转换为整数表示；`chr()` 则把整数码点转换回对应字符：

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

Problem (unicode1): 理解 Unicode (1 分)

- (a) `chr(0)` 返回哪个 Unicode 字符？

提交内容 (Deliverable): 用一句话回答。

- (b) 这个字符的字符串表示 `__repr__()` 与直接打印出来的表示有什么不同？

提交内容 (Deliverable): 用一句话回答。

- (c) 当这个字符出现在文本中时会发生什么？你可以在 Python 解释器里尝试下面这些表达式，并检查结果是否符合预期：

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

提交内容 (Deliverable): 用一句话回答。

2.2 Unicode 编码

Unicode 标准本身定义的是字符到码点（整数）的映射，但直接在 Unicode 码点上训练分词器并不现实：词表规模会非常大（约 15 万项），而且十分稀疏，因为许多字符极少出现。因此，我们使用 **Unicode 编码 (Unicode encoding)**，把一个 Unicode 字符转换为一个字节序列。

Unicode 标准定义了三种主要编码：UTF-8、UTF-16 和 UTF-32。其中 UTF-8 是互联网的主流编码，超过 98% 的网页使用 UTF-8。

在 Python 中，可以用 `encode()` 把 Unicode 字符串编码为 UTF-8；迭代一个 `bytes` 对象（例如调用 `list()`）即可访问底层字节值；再用 `decode()` 把 UTF-8 字节串解码回 Unicode 字符串：

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 取得底层字节值 (0 到 255 的整数)。
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129,
 171, 227, 129, 161, 227, 129, 175, 33]
>>> # 一个字节不一定对应一个 Unicode 字符!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

把 Unicode 码点转换成字节序列（例如采用 UTF-8），本质上是把一串码点——可理解为 21 位整数，其中有 159,801 个合法值——变成取值范围为 0 到 255 的字节值序列。长度为 256 的字节词表显然更容易处理。采用字节级分词时，也无需担心词表外标记（out-of-vocabulary, OOV），因为任意输入文本都必然可以表示为 0 到 255 的整数序列。

Problem (unicode2): Unicode 编码 (3 分)

- (a) 相比 UTF-16 或 UTF-32，为什么更适合在 UTF-8 编码后的字节上训练分词器？可以比较不同输入字符串在这几种编码下的结果。

提交内容 (Deliverable): 用 1-2 句话回答。

- (b) 考虑下面这个错误函数，它想把 UTF-8 字节串解码成 Unicode 字符串。它为什么错？给出一个会产生错误结果的输入字节串。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```


提交内容 (Deliverable): 给出一个会让该函数产生错误结果的字节串，并用一句话解释为什么函数错误。

(c) 给出一个无法解码成任何 Unicode 字符的两字节序列。

提交内容 (Deliverable): 给出一个例子，并用一句话解释。

2.3 子词分词

字节级分词虽然能消除词级分词器的 OOV 问题，却会让输入序列变得很长。一个有 10 个单词的句子，在词级语言模型里可能只有 10 个 token；在字符级模型中却可能需要 50 个甚至更多 token，具体取决于单词长度。更长的序列意味着模型每一步需要更多计算，同时也会在数据中引入更长程的依赖，使字节序列上的语言建模更困难。

子词分词 (subword tokenization) 位于词级分词与字节级分词之间。字节级分词器的初始词表有 256 个条目，对应字节值 0-255；子词分词器则以更大的词表换取更好的输入压缩率。例如，如果字节序列 `b'the'` 在原始训练文本中频繁出现，我们可以把它作为词表中的一个独立 token，从而把原本 3 个 token 的序列压缩成 1 个 token。

那么该如何选择加入词表的子词单元？Sennrich 等人 [3] 提议使用**字节对编码 (Byte-Pair Encoding, BPE)**，其源头是 P. Gage [5] 的压缩算法。BPE 会不断寻找出现频率最高的一对字节，把它们“合并”为一个新的、此前未使用的索引。它不断往词表中加入能够最大程度压缩输入序列的子词单元；如果一个完整单词在训练文本中出现足够频繁，它甚至可能最终成为一个独立的子词 token。

通过 BPE 构造词表的子词分词器通常称为 BPE 分词器。本作业实现**字节级 BPE**：词表中的条目要么是单个字节，要么是若干字节合并后的序列，从而同时获得“无 OOV”和“序列长度可控”这两个优点。构造 BPE 词表的过程通常称为“训练 BPE 分词器”。

2.4 BPE 分词器训练

BPE 训练大体分为三个主要步骤。

词表初始化

分词器词表是“字节串 token 到整数 ID”的一一映射。由于我们训练的是字节级 BPE，初始词表就是所有可能的单字节值，因此大小为 256。

预分词 (Pre-tokenization)

理论上，有了初始词表后，你可以直接统计整份文本中相邻字节对的出现次数，然后从最高频字节对开始合并。但这会非常昂贵，因为每执行一次合并，都可能需要重新扫描整个语料库。

此外，如果直接跨整个语料合并字节，还容易得到只在标点上不同的 token，例如 `dog!` 与 `dog.`。它们虽然语义高度相似，却会被赋予完全不同的 token ID。

因此，我们先对语料进行**预分词**。可以把它理解为一次粗粒度切分，用来高效统计字符对的频率。例如，单词 `text` 作为一个 pre-token 在语料中出现了 10 次，那么在统计相邻字符 `t` 和 `e` 时，可以直接给这对字符增加 10 次，而不必逐字扫描整个语料。由于这里训练的是字节级 BPE，每个 pre-token 最终表示为一串 UTF-8 字节。

Sennrich 等人最早的 BPE 实现只是按空白字符切分，即 `s.split(" ")`。这种方案仍能在基于 SentencePiece 的一些分词器中看到，例如 Llama 1 和 Llama 2。

现代分词器多数采用基于正则表达式的预分词器，这一做法来自 GPT-2 [A. Radford et al., 2019]。本作业使用下面这个稍经整理的 GPT-2 正则，来源于 tiktoken 的一个 PR:

```
>>> PAT = r"""(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

可以交互式地观察它如何切分文本：

```
>>> # 需要 `regex` 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

但在正式实现中，建议使用 `re.finditer`，这样在构建“pre-token 到出现次数”的映射时，不必把全部预分词结果都存进内存。

计算 BPE 合并

现在输入文本已经被分成 pre-token，并且每个 pre-token 又被表示为 UTF-8 字节序列。接下来就可以计算 BPE merges，也就是正式“训练”BPE 分词器。

在高层上，BPE 会反复统计所有相邻字节对，找到出现次数最多的一对，例如 (“A”，“B”)，然后把所有这类相邻出现替换为一个新 token “AB”。新 token 被加入词表。因此，训练后的最终词表大小等于初始词表大小 256，加上执行的 BPE merge 次数，再加上特殊 token 数量。

为了提高效率，训练时不考虑跨 pre-token 边界的字节对。²

若多个字节对频率并列最高，需要保证 tie-breaking 是确定性的：选择字典序更大的一对。例如，下列 4 对的频率相同时，应合并 ("BA", "A")：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

特殊 token

某些字符串（例如 `<|endoftext|>`）用来编码元数据，比如文档边界。编码文本时，我们通常希望把这些字符串作为**特殊 token (special token)**，永远不要拆成多个 token。比如，序列结束标记 `<|endoftext|>` 应始终对应一个单独整数 ID，这样模型生成到它时就知道应该停止。特殊 token 也必须加入词表，并拥有固定 token ID。

Sennrich 等人 [3] 的 Algorithm 1 给出了一个低效的 BPE 训练实现，基本就是上面这些步骤。作为练习，可以先实现一个简单版本，以检查自己是否真正理解 merge 过程。

Example (bpe example): BPE 训练示例

下面给出一个改写自 Sennrich 等人 [3] 的示意例子。语料为：

```
low low low low low
lower lower widest widest widest
```

²Sennrich 等人的原始 BPE 公式会显式加入 end-of-word token。字节级 BPE 不需要这样做，因为空格和标点本身就已经是字节词表的一部分，学习到的 merge 会自然反映单词边界。

```
newest newest newest newest newest newest
```

并且词表中有一个特殊 token: `<|endoftext|>`。

词表。初始词表包含特殊 token `<|endoftext|>` 和 256 个字节值。

预分词。为了突出 merge 过程，这个例子假设预分词只按空格切分。统计后得到：

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

可以方便地把它表示成 `dict[tuple[bytes, ...], int]`，例如 `{(l,o,w): 5, ...}`。注意，即使只有一个字节，在 Python 中它仍是 `bytes` 对象；Python 没有单独表示一个字节的 `byte` 类型，就像也没有单独的 `char` 类型一样。

合并。第一次统计所有连续字节对：

```
{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3,
 es: 9, st: 9, ne: 6, ew: 6}
```

`('e','s')` 与 `('s','t')` 的计数并列为 9，因此按照字典序 tie-break，选择更大的 `('s','t')`。合并后得到：

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}
```

第二轮中，`(e, st)` 以计数 9 成为最高频 pair，合并后：

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}
```

继续执行，可得到 merge 序列：

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e',
 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']
```

如果只执行 6 次合并，则 merges 为：

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']
```

词表元素将是 `<|endoftext|>`、256 个初始字节，以及 `st`, `est`, `ow`, `low`, `west`, `ne`。在该词表和 merge 规则下，`newest` 会被编码成 `[ne, west]`。

2.5 BPE 分词器训练实验

现在在 TinyStories 数据集上训练字节级 BPE 分词器。数据集获取方式见第 1 节。开始之前，建议先浏览 TinyStories 的原始文本，对数据内容有直观认识。

并行化预分词

预分词通常会成为主要瓶颈。可以使用 Python 内置的 `multiprocessing` 并行化它。具体建议是：把语料切成若干块，同时保证每个 chunk 的边界落在一个特殊 token 的起始位置。课程允许直接使用下列 starter code 获取 chunk 边界，再把工作分发给多个进程：

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种切分始终有效，因为 BPE 本来也不应该跨文档边界进行 merge。本作业中可以始终这样做；不必处理“超大语料完全不含 `<|endoftext|>`”这种极端情况。

预分词之前移除特殊 token

在用正则模式执行预分词 (`re.finditer`) 之前, 应先把全部特殊 token 从语料 (或每个 chunk) 中剥离。务必按特殊 token 切分, 以确保它所分隔的文本之间不会发生 merge。例如:

```
[Doc 1]<|endoftext|>[Doc 2]
```

应先按 `<|endoftext|>` 分成 [Doc 1] 和 [Doc 2], 再分别预分词。换言之, 特殊 token 在训练中定义**硬分段边界**, 但它自己不应该参与 merge 统计。

可以使用 `re.split`, 以 `"|".join(special_tokens)` 作为分隔正则, 并注意用 `re.escape` 转义特殊 token 内可能出现的 `|`。测试 `test_train_bpe_special_tokens` 会检查这一点。

优化 merge 步骤

前面示意例子的朴素实现很慢, 因为每次 merge 都会遍历所有 byte pair 重新找最高频项。事实上, 每次 merge 后, 只有**与被合并 pair 重叠的那些 pair** 的计数会变化。因此, 可以提前为所有 pair 建立索引, 只增量更新受影响的计数, 而不是每轮重新遍历整套字节对。这样的缓存策略能带来显著提速。但要注意: BPE 的 merge 部分在 Python 中并不容易直接并行化。

低资源提示: Profiling

使用 `cProfile`、`py-spy` 等 profiling 工具定位实现中的真正瓶颈, 然后把优化精力集中在瓶颈上。

低资源提示: “Downscaling”

不要一上来就在完整 TinyStories 语料上训练。建议先用很小的子集作为“调试数据集 (debug dataset)”, 例如 TinyStories validation set: 约 22K 篇文档, 而完整训练集有约 2.12M 篇。这体现了一条通用开发原则: 尽可能缩小数据集、模型等规模来加速迭代。调试配置也不能太小——它仍应保留完整配置中的主要瓶颈, 这样你在小规模上做出的优化才能推广; 同时又不能大到每次运行都耗时很久。

Problem (train_bpe): 训练 BPE 分词器 (15 分)

提交内容 (Deliverable): 编写一个函数: 给定输入文本文件路径, 训练一个字节级 BPE 分词器。函数至少应支持以下参数:

输入:

- `input_path: str`: BPE 训练数据文本文件路径;
- `vocab_size: int`: 正整数, 规定最终最大词表大小; 它包括初始 256 字节、merge 生成的新 token, 以及特殊 token;
- `special_tokens: list[str]`: 需要加入词表的特殊字符串。训练时把它们视为禁止跨越的硬边界, 但不把它们计入 merge 统计。

函数应返回:

输出:

- `vocab: dict[int, bytes]`: 词表, 即 token ID (int) 到 token 字节串 (bytes) 的映射;
- `merges: list[tuple[bytes, bytes]]`: 训练得到的 merge 列表。每一项 (`<token1>`, `<token2>`) 表示这两个 token 被合并; 列表顺序应与 merge 创建顺序一致。

要运行课程测试，先实现 `adapter adapters.run_train_bpe`，然后运行：

```
uv run pytest tests/test_train_bpe.py
```

实现应通过全部测试。

可选挑战 (工作量可能很大)：可以用系统语言实现训练中的关键部分，例如 C++ (考虑 `cppy` 或 `nanobind`) 或 Rust (PyO3)。如果这样做，要清楚哪些操作会发生拷贝、哪些可以直接读取 Python 内存，并提供构建说明，或者确保仅通过 `pyproject.toml` 即可构建。还要注意，GPT-2 正则引擎在多数正则引擎中支持不佳或很慢；课程确认 Oniguruma 性能尚可并支持 `negative lookahead`，而 Python 的 `regex` 包甚至可能更快。

Problem (train_bpe_tinystories): 在 TinyStories 上训练 BPE (2 分)

- (a) 在 TinyStories 上训练字节级 BPE，最大词表大小设为 10,000，并把 TinyStories 使用的特殊 token `<|endoftext|>` 加入词表。把最终词表和 merge 列表序列化到磁盘，供后续检查。训练耗时和内存占用是多少？词表中最长的 token 是什么？它是否合理？

资源要求：不超过 30 分钟 (无 GPU)，内存不超过 30 GB。

提示：利用预分词阶段的 `multiprocessing`，以及以下两点，BPE 训练应能做到 2 分钟以内：`<|endoftext|>` 在数据文件中划分文档；并且在应用 BPE merges 之前先对它进行特殊处理。

提交内容 (Deliverable)：用 1-2 句话回答。

- (b) 对代码进行 `profiling`。分词器训练中哪一部分最耗时？

提交内容 (Deliverable)：用 1-2 句话回答。

接下来在 OpenWebText 上训练字节级 BPE。与前面一样，建议先查看数据集内容。

Problem (train_bpe_expts_owt): 在 OpenWebText 上训练 BPE (2 分)

- (a) 在 OpenWebText 上训练字节级 BPE，最大词表大小为 32,000。将得到的词表和 merge 列表序列化到磁盘。词表中最长 token 是什么？它是否合理？

资源要求：不超过 12 小时 (无 GPU)，内存不超过 100 GB。

提交内容 (Deliverable)：用 1-2 句话回答。

- (b) 比较在 TinyStories 与 OpenWebText 上训练得到的分词器。

提交内容 (Deliverable)：用 1-2 句话回答。

2.6 BPE 分词器：编码与解码

前一部分已经实现了从输入文本训练 BPE、获得词表与 merge 列表的函数。现在要实现一个真正的 BPE 分词器：加载给定词表和 merges，并据此在文本与 token ID 之间进行双向转换。

2.6.1 编码文本

BPE 编码与训练词表的过程非常相似，主要分为两步。

步骤 1：预分词。先预分词，再把每个 pre-token 表示成 UTF-8 字节序列，与 BPE 训练阶段相同。随后只在每个 pre-token 内部合并字节，不允许跨 pre-token 边界 merge。

步骤 2：按顺序应用 merges。取得训练阶段生成的 merge 序列，并严格按照创建顺序应用到每个 pre-token。

Example (bpe_encoding): BPE 编码示例

假设输入字符串是 'the cat ate', 词表为:

```
{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't',
 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}
```

训练得到的 merges 为:

```
[(b't', b'h'), (b' ', b'c'), (b' ', b'a'),
 (b'th', b'e'), (b' a', b't')]
```

预分词先得到:

```
['the', ' cat', ' ate']
```

对第一个 pre-token 'the', 初始表示是 [b't', b'h', b'e']。列表中第一个可用 merge 为 (b't', b'h'), 因此得到 [b'th', b'e']; 继续从 merge 列表中按顺序寻找, 下一项可应用的是 (b'th', b'e'), 于是得到 [b'the']。此后没有更多可用 merge, 因此整数 ID 序列为 [9]。同理, ' cat' 最终变成 [b' c', b'a', b't'], 对应 [7,1,5]; ' ate' 变成 [b' at', b'e'], 对应 [10,3]。因此完整字符串最终编码为:

```
[9, 7, 1, 5, 10, 3]
```

特殊 token

编码文本时, 分词器应正确处理用户在构造分词器时传入的特殊 token。

内存方面的考虑

如果要 tokenize 一个大到无法完整放入内存的文本文件, 就必须把它分成可管理的 chunk, 逐块处理, 这样内存复杂度保持为常数, 而不是随文本长度线性增长。切块时必须保证一个 token 不会跨 chunk 边界, 否则分块编码结果将不同于“整个序列一次性放进内存”的朴素编码结果。

2.6.2 解码文本

把 token ID 序列解码回原始文本很直接: 查词表得到每个 ID 对应的字节串, 把字节串全部拼接起来, 再解码成 Unicode 字符串。

但输入 ID 序列不保证对应一个合法 Unicode 字符串, 因为用户可以任意组合 token ID。如果拼接出的字节不是合法 Unicode, 应使用官方 Unicode 替换字符 U+FFFD 替换损坏字节。bytes.decode 的 errors 参数控制错误处理方式, 设置 errors='replace' 会自动插入替换字符。³

Problem (tokenizer): 实现 Tokenizer (15 分)

提交内容 (Deliverable): 实现一个 Tokenizer 类。给定词表与 merges, 它应能把文本编码成整数 ID, 也能把整数 ID 解码回文本; 还应支持用户提供的特殊 token, 如果特殊 token 尚不在词表中, 则将其追加到词表。

推荐接口:

³可参见 [https://en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。


```
def __init__(self, vocab, merges, special_tokens=None)
```

从给定词表、merge 列表以及可选特殊 token 构造分词器，参数为：

- vocab: dict[int, bytes]
- merges: list[tuple[bytes, bytes]]
- special_tokens: list[str] | None = None

```
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
```

类方法：从序列化的词表与 merges 文件构造并返回一个 Tokenizer，文件格式应与你的 BPE 训练代码输出一致。额外参数：

- vocab_filepath: str
- merges_filepath: str
- special_tokens: list[str] | None = None

```
def encode(self, text: str) -> list[int]
```

把输入文本编码为 token ID 序列。

```
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]
```

给定字符串 iterable（例如 Python 文件句柄），返回一个惰性产生 token ID 的生成器。这样才能内存高效地 tokenize 无法整体载入内存的大文件。

```
def decode(self, ids: list[int]) -> str
```

把 token ID 序列解码成文本。

实现 adapter `adapters.get_tokenizer`，然后运行：

```
uv run pytest tests/test_tokenizer.py
```

实现应通过全部测试。

2.7 分词器实验

Problem (tokenizer_experiments): 分词器实验 (4 分)

- (a) 分别从 TinyStories 与 OpenWebText 中抽取 10 篇文档。使用此前训练的 TinyStories 分词器（10K 词表）和 OpenWebText 分词器（32K 词表）把它们编码成整数 ID。两种分词器各自的压缩率（bytes/token）是多少？

提交内容 (Deliverable): 用 1-2 句话回答。

- (b) 如果用 TinyStories 分词器去 tokenize OpenWebText 样本，会发生什么？比较压缩率，或者定性描述结果。

提交内容 (Deliverable): 用 1-2 句话回答。

- (c) 估计你的分词器吞吐量，例如 bytes/s。若要 tokenize Pile 数据集（825 GB 文本），大约需要多久？

提交内容 (Deliverable): 用 1-2 句话回答。

- (d) 分别用 TinyStories 与 OpenWebText 分词器编码各自的训练集和开发集，得到整数 token ID 序列。后面训练语言模型时会使用这些数据。建议把 token ID 序列序列化成 NumPy 的 `uint16` 数组。为什么 `uint16` 是合适选择？

提交内容 (Deliverable): 用 1-2 句话回答。

3 Transformer 语言模型架构

语言模型接收一批整数 token ID 序列作为输入，即形状为 $(\text{batch_size}, \text{sequence_length})$ 的 `torch.Tensor`，并返回词表上的一批归一化概率分布，即形状为 $(\text{batch_size}, \text{sequence_length}, \text{vocab_size})$ 的 PyTorch 张量。对于输入序列中的每个位置，这个分布预测下一个 token。

训练语言模型时，我们用这些 next-token 预测与真实下一个 token 之间的差异计算交叉熵损失。推理生成文本时，则取最后一个时间步的 next-token 概率分布，用它产生序列中的下一个 token，例如取最大概率项或按分布采样；把生成 token 追加到输入序列后重复该过程。

本部分将从零构建这个 Transformer 语言模型。我们先从高层结构开始，再逐步深入每个组件。

3.1 Transformer LM

给定一串 token ID，Transformer LM 首先用输入 embedding 把 token ID 转换成稠密向量，然后依次通过 `num_layers` 个 Transformer block，最后应用一个可学习的线性投影，也称“输出 embedding”或“LM head”，得到对下一个 token 的 logits。图 1 给出整体结构。

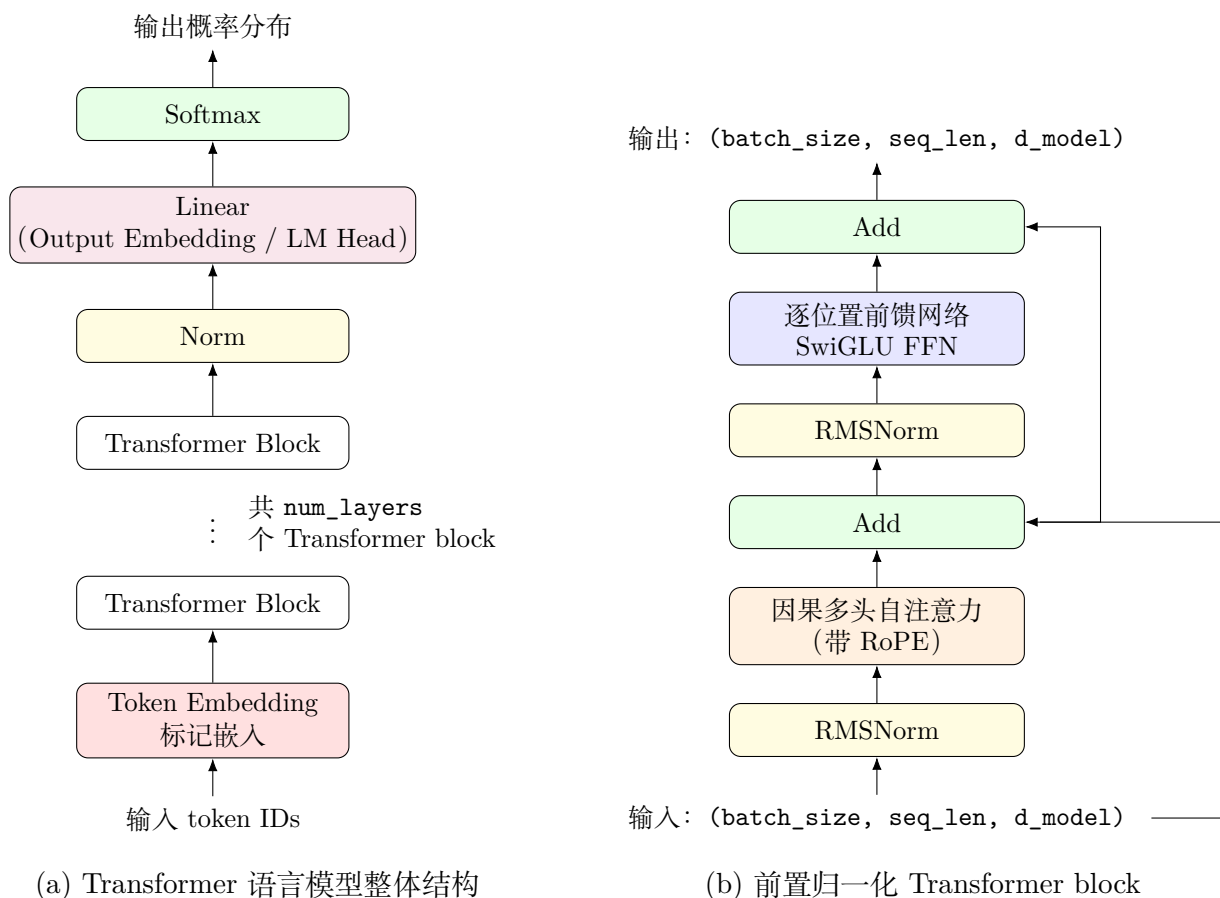


图 1: 本作业 Transformer LM 的整体结构与 pre-norm Transformer block。

Token Embeddings

第一步，Transformer 把一批 token ID 序列嵌入为一批向量，这些向量编码了 token 身份信息。

具体来说，token embedding 层接收形状为 $(\text{batch_size}, \text{sequence_length})$ 的整数张量，输出形状为 $(\text{batch_size}, \text{sequence_length}, d_{\text{model}})$ 的向量序列。

Pre-norm Transformer Block

Embedding 之后，激活值会经过若干结构完全相同的神经网络层。标准 decoder-only Transformer LM 由 `num_layers` 个相同层组成，通常称为 Transformer “block”。每个 block 的输入和输出形状都是 $(\text{batch_size}, \text{sequence_length}, d_{\text{model}})$ 。其中，自注意力负责跨序列聚合信息，前馈网络负责对每个位置进行非线性变换。

经过 `num_layers` 个 block 后，我们把最终激活转成词表上的分布。

本作业实现前置归一化 (pre-norm) Transformer block (第 3.4 节详细说明)。由于采用 pre-norm，还需要在最后一个 Transformer block 之后额外进行一次层归一化，保证最终输出尺度合适。之后使用一个标准的可学习线性变换，把 block 输出转换成 next-token logits (例如见 Radford 等人 [7] 的 Equation 2)。

3.2 补充说明：Batching、Einsum 与高效计算

在 Transformer 中，同一种计算经常要同时应用到大量“类似 batch 的维度”上，例如：

- **batch 元素**：同一个 Transformer 前向传播应用于 batch 中每个样本；
- **序列长度**：RMSNorm、feed-forward 等“逐位置 (position-wise)”运算对序列的每个位置执行同样的计算；
- **注意力头**：多头注意力把每个 attention head 当成一个额外 batch 维度，独立执行注意力。

我们希望用一种既能充分利用 GPU，又容易阅读和理解的方式表达这些批量运算。许多 PyTorch 算子允许张量前面带额外“batch-like”维度，然后自动在这些维度上重复或广播运算。

例如，设数据张量 D 的形状为 $(\text{batch_size}, \text{sequence_length}, d_{\text{model}})$ ，矩阵 A 的形状为 $(d_{\text{model}}, d_{\text{model}})$ 。表达式 $D @ A$ 会对前两个维度进行 batch 化，执行高效的 batched matrix multiplication。

因此，编写函数时最好默认输入可能带有任意额外 batch-like 维度，并把这些维度放在 PyTorch shape 的前部。若只依赖 `view`、`reshape`、`transpose` 来整理张量形状，代码很容易变得难读，而且很难从代码本身看出每个维度代表什么。

更易读的方法是使用 `torch.einsum` 的 Einstein summation notation，或者使用与框架无关的 `einops` / `einx`。两个关键操作是：

- **einsum**：在任意输入维度上执行张量缩并 (tensor contraction)；
- **rearrange**：重排、合并与拆分任意维度。

实际上，机器学习中的大多数操作都可以看成“维度整理 + 张量缩并 + 偶尔的逐元素非线性函数”。因此，`einsum` 往往能让代码更可读、更灵活。

课程强烈推荐掌握并使用 `einsum`。此前没接触过 `einsum` 的同学可以先使用 `einops`；已经熟悉

einops 的同学可以进一步学习更通用的 einx。⁴

Example (einstein_example1): 用 einops.einsum 进行批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## 基础实现
Y = D @ A.T
# 很难仅从代码判断输入/输出 shape 及其语义。
# D 与 A 可以是什么形状？某些输入是否会出现意外广播？

## Einsum 具有“自文档化”特性，并且更稳健
#
#           D           A           ->           Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## D 还可以有任意数量的前导 batch 维度，而 A 的轴仍受约束。
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

Example (einstein_example2): 用 einops.rearrange 做广播运算

假设有一批图像，希望每张图像生成 10 个不同亮度的版本：

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## 先 reshape 再乘法
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## 或者一步完成
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

Example (einstein_example3): 用 einops.rearrange 做像素混合

假设一批图像张量的形状为 (batch, height, width, channel)，我们希望对所有像素执行线性变换，但每个 channel 独立变换。线性变换矩阵 B 的形状为 (height * width, height * width)。

```
channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## 用 view + transpose 把图像整理成适合像素混合的形状
channels_last_flat = channels_last.view(
```

⁴einops 的生态和测试更成熟，einx 相对没有那么久经验证；如果遇到限制或 bug，可以退回 einops 配合普通 PyTorch。

```

        -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
    )
    channels_first_flat = channels_last_flat.transpose(1, 2)
    channels_first_flat_transformed = channels_first_flat @ B.T
    channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
    channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)

    ## 使用 einops
    height = width = 32
    channels_first = rearrange(
        channels_last,
        "batch height width channel -> batch channel (height width)"
    )
    channels_first_transformed = einsum(
        channels_first, B,
        "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
    )
    channels_last_transformed = rearrange(
        channels_first_transformed,
        "batch channel (height width) -> batch height width channel",
        height=height, width=width
    )

    ## 如果想更激进, 也可以一次用 einx.dot 完成
    height = width = 32
    channels_last_transformed = einx.dot(
        "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
        "-> batch row_out col_out channel",
        channels_last, B,
        col_in=width, col_out=width
    )

```

普通实现可以靠前后加注说明输入输出 shape, 但这样笨重且容易出错。einsum notation 的优势在于: **文档本身就是实现的一部分**。

Einsum 可以处理任意输入 batch 维度, 而且具有很强的自文档化能力。对于剩余张量, 也可以考虑使用类型提示, 例如 `jaxtyping` (虽然名字里有 JAX, 但并不只适用于 JAX)。Assignment 2 会进一步讨论 einsum 的性能影响; 在这里先记住, 它们几乎总比大量手写 reshape/transpose 更适合作为清晰的默认表达方式。

3.2.1 数学记号与内存顺序

许多机器学习论文使用**行向量**记号, 这与 NumPy / PyTorch 默认采用的 row-major (行主序) 内存排列比较契合。对行向量而言, 线性变换写为:

$$y = xW^T, \quad (1)$$

其中 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $x \in \mathbb{R}^{1 \times d_{\text{in}}}$ 。这样只需扩展 x 的最外层维度, 就可以把单向量输入替换为 batch 矩阵 $X \in \mathbb{R}^{\text{batch} \times d_{\text{in}}}$ 。

在线性代数中，更常见的是**列向量**记号：

$$y = Wx, \quad (2)$$

其中同样有 row-major 的 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ ，而 $x \in \mathbb{R}^{d_{\text{in}}}$ 被视为列向量。若要在这种记号下批量化输入，batch 维度就会出现在最后，形成 $\tilde{X} \in \mathbb{R}^{d_{\text{in}} \times \text{batch}}$ 。

本作业的数学推导主要采用列向量记号，因为数学教材通常如此书写。实现时要记住：如果直接使用普通矩阵乘法记号，为了适应 PyTorch 的 row-major 张量组织，往往需要像式 (1) 那样对权重做转置。如果使用 einsum，只要正确标注每个轴，通常就不会成为问题。顺带一提，Matlab、Julia 和 Fortran 默认使用 column-major，而 Python 及相关数值库继承了 C 风格的 row-major。

3.3 基础构件：Linear 与 Embedding 模块

3.3.1 参数初始化

高效训练神经网络往往依赖合理的参数初始化；糟糕初始化可能导致梯度消失或梯度爆炸。Pre-norm Transformer 对初始化相对鲁棒，但初始化仍会显著影响训练速度与收敛。本作业篇幅已经很长，更细的初始化原理会留到 Assignment 3，这里直接给出一组近似且通常有效的初始化方案：

- **Linear 权重：**

$$W \sim \mathcal{N}\left(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}}\right),$$

并在 $[-3\sigma, 3\sigma]$ 截断；

- **Embedding：** $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ ，并在 $[-3, 3]$ 截断；
- **RMSNorm：** 初始化为全 1 向量 $\mathbf{1}$ 。

截断正态分布可使用 `torch.nn.init.trunc_normal_`。

3.3.2 Linear 模块

Linear 是 Transformer 和一般神经网络的基本构件。首先实现自己的 Linear 类，继承自 `torch.nn.Module`，执行：

$$y = Wx. \quad (3)$$

与多数现代 LLM 一样，这里不使用 bias。

Problem (linear): 实现 Linear 模块 (1 分)

提交内容 (Deliverable)： 实现一个继承 `torch.nn.Module` 的 Linear 类，执行线性变换。接口应尽量与 PyTorch 内置 `nn.Linear` 一致，但没有 `bias` 参数或 `bias parameter`。

推荐接口：

```
def __init__(self, in_features, out_features, device=None, dtype=None)
```

其中：

- `in_features: int`：输入最后一维大小；
- `out_features: int`：输出最后一维大小；
- `device: torch.device | None = None`：参数存储设备；

- `dtype: torch.dtype | None = None`: 参数数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

对输入执行线性变换。

请确保：继承 `nn.Module`；调用父类构造函数；把参数以 W （而不是 W^T ）的形式存储在 `nn.Parameter` 中；不得使用 `nn.Linear` 或 `nn.functional.linear`。初始化使用前述设置与 `torch.nn.init.trunc_normal_`。

实现 `adapters.run_linear`。该 adapter 应把测试提供的权重加载到你的 `Linear` 中，可以使用 `Module.load_state_dict`。随后运行：

```
uv run pytest -k test_linear
```

3.3.3 Embedding 模块

Transformer 第一层 embedding 把整数 token ID 映射到 d_{model} 维向量空间。本作业自行实现一个继承 `torch.nn.Module` 的 `Embedding`，因此不得调用 `nn.Embedding`。

Forward 应用形状为 `(batch_size, sequence_length)` 的 `torch.LongTensor` token ID 去索引形状为 `(vocab_size, d_model)` 的 embedding matrix，从而为每个 token ID 取出对应向量。

Problem (embedding): 实现 Embedding 模块 (1 分)

提交内容 (Deliverable): 实现继承 `torch.nn.Module` 的 `Embedding` 类，完成 embedding lookup。接口应尽量与 PyTorch 的 `nn.Embedding` 一致。

推荐接口：

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

参数：

- `num_embeddings: int`: 词表大小；
- `embedding_dim: int`: embedding 维度，即 d_{model} ；
- `device: torch.device | None = None`: 参数设备；
- `dtype: torch.dtype | None = None`: 参数数据类型。

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

查找输入 token ID 对应的 embedding vector。

请确保：继承 `nn.Module`；调用父类构造函数；embedding matrix 是 `nn.Parameter`；矩阵最后一维是 d_{model} ；不得调用 `nn.Embedding` 或 `nn.functional.embedding`。初始化仍使用前述方案与 `trunc_normal_`。

实现 `adapters.run_embedding` 后运行：

```
uv run pytest -k test_embedding
```

3.4 Pre-Norm Transformer Block

每个 Transformer block 有两个子层：多头自注意力（multi-head self-attention）和逐位置前馈网络（position-wise feed-forward network）[Vaswani et al., 2017, Section 3.1]。

原始 Transformer 在两个子层外都使用 residual connection，随后再做 layer normalization，因此通常称为 **post-norm Transformer**：归一化发生在子层输出端。后续多项工作发现，把 layer normalization 从每个子层的输出移动到输入端，并在最终 Transformer block 后额外加入一次 layer norm，可以提高训练稳定性 [Nguyen et al., 2019; Xiong et al., 2020]。这就是图 1(b) 所示的 **pre-norm** 结构。

每个子层计算结果通过残差连接加回子层输入。直观上，pre-norm 让从 input embedding 一直到最终 Transformer 输出之间存在一条“干净的 residual stream”，归一化层不会直接截断这条主干路径，因此被认为有助于梯度流动。如今 GPT-3、LLaMA、PaLM 等语言模型普遍使用 pre-norm，本作业也实现这一变体。

3.4.1 均方根层归一化 (RMSNorm)

原始 Transformer 使用 Layer Normalization [Ba et al., 2016]。本作业追随 LLaMA 的做法，使用 Root Mean Square Layer Normalization，即 RMSNorm [Zhang and Sennrich, 2019]。

给定激活向量 $a \in \mathbb{R}^{d_{\text{model}}}$ ，RMSNorm 对第 i 个分量的缩放为：

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \quad \text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}. \quad (4)$$

其中 g_i 是可学习的 gain parameter，总共有 d_{model} 个； ε 是数值稳定超参数，常设为 10^{-5} 。

由于平方操作可能溢出，应先把输入 upcast 到 `torch.float32`，计算完成后再 cast 回输入的原 dtype：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# 在这里实现 RMSNorm
...
result = ...

# 返回原始 dtype
return result.to(in_dtype)
```

Problem (rmsnorm): 实现 RMSNorm (1 分)

提交内容 (Deliverable): 把 RMSNorm 实现为 `torch.nn.Module`。推荐接口：

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

参数： d_{model} 、用于数值稳定的 ε 、参数设备与 dtype。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

输入形状为 `(batch_size, sequence_length, d_model)`，返回相同形状。

注意：归一化前应把输入转换成 `torch.float32`，最后再转回原 dtype。实现

`adapters.run_rmsnorm`, 运行:

```
uv run pytest -k test_rmsnorm
```

3.4.2 逐位置前馈网络

原始 Transformer 的 FFN 是两次线性变换，中间使用 ReLU，且隐藏层维度通常为输入维度的 4 倍。现代 LLM 通常做两点修改：更换激活函数，并引入门控机制。本作业实现 Llama 3、Qwen 2.5 等模型采用的 **SwiGLU**，它把 SiLU（也称 Swish）与 Gated Linear Unit (GLU) 结合起来。同时遵循 PaLM、LLaMA 以来的主流做法，不在线性层使用 bias。

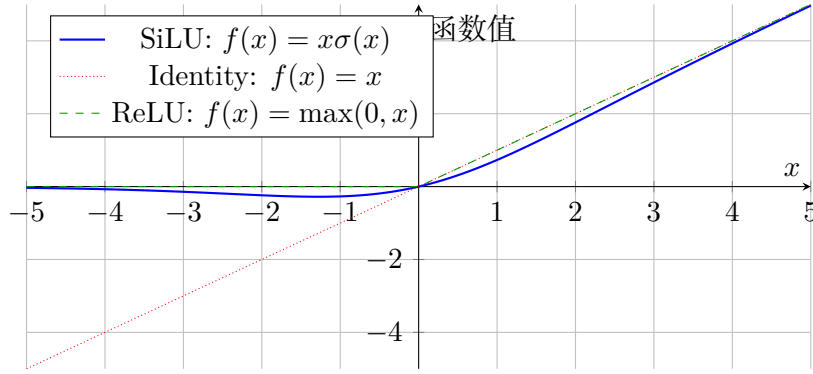


图 2: SiLU (Swish) 与 ReLU 激活函数的比较。

SiLU / Swish 定义为:

$$\text{SiLU}(x) = x\sigma(x) = \frac{x}{1 + e^{-x}}. \quad (5)$$

如图 2, SiLU 与 ReLU 的整体趋势类似, 但在 $x = 0$ 附近平滑可导。

Gated Linear Unit (GLU) 最初由 Dauphin 等人 [19] 定义为 “一个经过 sigmoid 的线性变换” 与 “另一个线性变换” 的逐元素乘积:

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

其中 $\odot$ 表示逐元素乘法。原论文的动机之一是：在保持非线性能力的同时，为梯度提供更直接的线性通路，从而缓解深层网络的梯度消失问题。

把 SiLU 与 GLU 结合得到本作业采用的 SwiGLU:

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2 (\text{SiLU}(W_1 x) \odot W_3 x), \quad (7)$$

其中

$$x \in \mathbb{R}^{d_{\text{model}}}, \quad W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \quad W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}.$$

经典设置是

$$d_{\text{ff}} \approx \frac{8}{3} d_{\text{model}},$$

工程实现中可以把它四舍五入到附近的 64 倍数，以提高硬件效率。

Shazeer [20] 首先系统比较了把 SiLU/Swish 与 GLU 组合的做法，并发现 SwiGLU 在语言建模中优于 ReLU 和不带 gating 的 SiLU baseline。本作业后面会直接对比 SwiGLU 与 SiLU。作者在论文中也留下了一句著名的经验主义式评论：他们并没有为这些结构为何有效给出理论解释，而

是把成功“归因于神意”。

Problem (positionwise_feedforward): 实现逐位置前馈网络 (2 分)

提交内容 (Deliverable): 实现由 SiLU 与 GLU 组成的 SwiGLU feed-forward network。

在这一题中，为了数值稳定，可以直接使用 `torch.sigmoid`。实现中令 $d_{\text{ff}} \approx \frac{8}{3}d_{\text{model}}$ ，同时确保内层维度是 64 的倍数，以更好利用硬件。

实现 `adapters.run_swiglu` 后运行：

```
uv run pytest -k test_swiglu
```

3.4.3 相对位置嵌入：RoPE

为了向模型注入位置信息，本作业实现**旋转位置嵌入 (Rotary Position Embeddings, RoPE)** [Su et al., 2021]。

对于位置 i 上的 query token，设

$$q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d.$$

我们应用一个按二维分量对分块的旋转矩阵 R^i ：

$$q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}.$$

对第 k 个二维 pair $q_{2k-1:2k}^{(i)}$ ，旋转角为

$$\theta_{i,k} = \frac{i}{\Theta(2k-2)/d}, \quad k \in \{1, \dots, d/2\},$$

其中 Θ 是常数。因此 R^i 可以看成是一个 $d \times d$ 的 block-diagonal matrix，第 k 个 2×2 block 为：

$$R_k^i = \begin{pmatrix} \cos \theta_{i,k} & -\sin \theta_{i,k} \\ \sin \theta_{i,k} & \cos \theta_{i,k} \end{pmatrix}. \quad (8)$$

完整矩阵为：

$$R^i = \begin{pmatrix} R_1^i & 0 & 0 & \cdots & 0 \\ 0 & R_2^i & 0 & \cdots & 0 \\ 0 & 0 & R_3^i & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_{d/2}^i \end{pmatrix}, \quad (9)$$

其中 0 表示 2×2 零矩阵。

当然，工程实现没必要显式构造完整 $d \times d$ 矩阵，而应利用其分块结构直接旋转成对分量。因为我们只关心同一序列内的相对旋转， $\cos \theta_{i,k}$ 与 $\sin \theta_{i,k}$ 可以跨 layer、跨 batch 重用。可以让所有层共享一个 RoPE 模块，在 `__init__` 阶段预计算二维的 sin/cos buffer，再用 `self.register_buffer` (并设置 `persistent=False`) 注册，而不是使用 `nn.Parameter`，因为这些值并不需要学习。

同样的旋转也应用到 key $k^{(j)}$ ，只是使用位置 j 对应的 R^j 。RoPE 层本身没有可学习参数。

Problem (rope): 实现 RoPE (2 分)

提交内容 (Deliverable): 实现一个 RotaryPositionalEmbedding 类, 把 RoPE 应用到输入张量。

推荐接口:

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

其中 θ 对应 Θ , d_k 是 query/key 维度, max_seq_len 是最大输入序列长度, device 为 buffer 存储设备。

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

输入形状为 $(\dots, \text{seq_len}, d_k)$, 输出同形状; 应允许任意数量 batch 维度。token_positions 的形状是 $(\dots, \text{seq_len})$, 给出序列维度上每个 x 的 token position。你应使用这些 position 去切片可能预计算好的 cos/sin buffer。

完成 `adapters.run_rope`, 运行:

```
uv run pytest -k test_rope
```

3.4.4 缩放点积注意力

接下来实现 Vaswani 等人 [8, Section 3.2.1] 的 scaled dot-product attention。首先需要 softmax, 它把未归一化分数向量转换为归一化分布:

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (10)$$

若 v_i 很大, $\exp(v_i)$ 可能溢出成 `inf`, 随后产生 $\text{inf}/\text{inf} = \text{NaN}$ 。Softmax 对“所有输入加同一个常数 c ”不变, 因此通常把 v 的最大元素从所有元素中减掉, 使新向量最大值为 0, 从而显著提高数值稳定性。

Problem (softmax): 实现 softmax (1 分)

提交内容 (Deliverable): 编写一个对张量应用 softmax 的函数。函数接收一个 tensor 和一个维度 i , 对输入张量第 i 维执行 softmax。输出 shape 与输入相同, 但第 i 维变为归一化概率分布。

必须使用“在第 i 维减去最大值”的数值稳定技巧。完成 `adapters.run_softmax` 并运行:

```
uv run pytest -k test_softmax_matches_pytorch
```

注意力操作定义为:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (11)$$

其中 $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, $V \in \mathbb{R}^{m \times d_v}$ 。这里 Q, K, V 都是该操作的输入, 不要把它们误认为可学习参数矩阵。

Masking。有时需要屏蔽注意力中的某些连接。Mask 形状为 $M \in \{\text{True}, \text{False}\}^{n \times m}$; 第 i 行告诉 query i 可以关注哪些 key。按照本作业约定 (虽然稍微反直觉), $M_{ij} = \text{True}$ 表示 query i 允许关注 key j , 而 `False` 表示不允许。换言之, `True` 的 (i, j) 位置允许“信息流动”。例如 `[[True, True, False]]` 表示唯一 query 只能关注前两个 key。

与分别对许多子序列计算 attention 相比, masking 更高效。实现时, 在 softmax 之前的分数 $QK^\top/\sqrt{d_k}$ 中, 把 mask 为 False 的位置加上 $-\infty$ 即可。

Problem (scaled_dot_product_attention): 实现缩放点积注意力 (5 分)

提交内容 (Deliverable): 实现 scaled dot-product attention。Query 与 key 的形状应支持:

$(\text{batch_size}, \dots, \text{seq_len}, d_k)$

value 的形状应支持:

$(\text{batch_size}, \dots, \text{seq_len}, d_v)$

其中 $\dots$ 表示任意数量额外 batch-like 维度。输出形状为:

$(\text{batch_size}, \dots, \text{seq_len}, d_v)$.

还要支持用户可选传入一个 $(\text{seq_len}, \text{seq_len})$ 的 boolean mask。所有 mask=True 的位置对应的注意力概率应共同归一化为 1; mask=False 的位置概率应为 0。

实现 `adapters.run_scaled_dot_product_attention`。测试命令:

```
uv run pytest -k test_scaled_dot_product_attention
uv run pytest -k test_4d_scaled_dot_product_attention
```

前者测试三阶输入, 后者测试四阶输入。

3.4.5 因果多头自注意力

多头自注意力按 Vaswani 等人 [8, Section 3.2.2] 定义:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h), \quad (12)$$

其中

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i). \quad (13)$$

Q_i, K_i, V_i 是 Q, K, V 在 embedding 维度上切出的第 i 个 head slice, query/key 每个 slice 大小为 d_k , value slice 大小为 d_v 。

于是 multi-head self-attention 写为:

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x). \quad (14)$$

可学习参数为:

$$W_Q, W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}, \quad W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}, \quad W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}.$$

因为 Q, K, V 会在 multi-head 操作中按 head 切片, 所以也可以把 W_Q, W_K, W_V 理解为在输出维度上包含每个 head 各自的一段投影。正常实现时, query、key、value 三个投影总共只需要 3 次矩阵乘法。⁵

⁵进阶目标: 把 Q、K、V 三个投影合并到一个权重矩阵, 从而只做一次矩阵乘法。

因果掩码 (Causal masking) 实现必须阻止模型看到未来 token。假设输入是 $t_1, \dots, t_n$ ，而要预测前缀 $t_1, \dots, t_i$ 后的下一个 token，那么模型不能访问 $t_{i+1}, \dots, t_n$ 的表示，因为推理生成时这些未来 token 根本还不存在；如果训练时能看到它们，就会泄漏真实 next-token 信息，使语言模型预训练目标失去意义。

朴素做法是对一个长度 n 的序列分别运行 n 次不同前缀的 attention，但这很浪费。更高效的做法是**因果注意力掩码**：位置 i 只允许关注所有 $j \leq i$ 的位置。可以用 `torch.triu` 或广播后的索引比较构造 mask，并复用上一节已支持 mask 的 scaled dot-product attention。

应用 RoPE RoPE 应用于 query 和 key，而**不应用于 value**。Head 维度应作为 batch 维度处理，因为各个 attention head 独立执行注意力。因此每个 head 的 query/key 都应用同样的 RoPE 旋转规则。

Problem (multihead_self_attention): 实现因果多头自注意力 (5 分)

提交内容 (Deliverable): 把 causal multi-head self-attention 实现为 `torch.nn.Module`。至少接收：

- `d_model: int`: Transformer block 输入维度；
- `num_heads: int`: multi-head self-attention 的 head 数量。

按照 Vaswani 等人 [8]，设置：

$$d_k = d_v = \frac{d_{\text{model}}}{h}.$$

实现 `adapters.run_multihead_self_attention` 后运行：

```
uv run pytest -k test_multihead_self_attention
```

3.5 完整 Transformer LM

现在开始组装 Transformer block。每个 block 包含两个子层：multi-head self-attention 与 SwiGLU feed-forward。每个子层的顺序都是：先 RMSNorm，再执行主要操作（MHA 或 FFN），最后把结果通过 residual connection 加回输入。

例如，第一个子层从输入 x 得到输出 y ：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \quad (15)$$

Problem (transformer_block): 实现 Transformer block (3 分)

按照第 3.4 节和图 1(b) 实现 pre-norm Transformer block。至少接收：

- `d_model: int`: block 输入维度；
- `num_heads: int`: MHA head 数；
- `d_ff: int`: 逐位置 FFN 内层维度。

实现 adapter `adapters.run_transformer_block`，运行：

```
uv run pytest -k test_transformer_block
```

提交内容 (Deliverable): 能通过测试的 Transformer block 代码。

接下来按照图 1(a) 组装完整模型：input embedding $\rightarrow$ `num_layers` 个 Transformer blocks $\rightarrow$ 最终 layer norm $\rightarrow$ LM head，得到词表上的未归一化分数 logits。

Problem (transformer_lm): 实现 Transformer LM (3 分)

实现第 3.1 节描述的 Transformer language model。除了 Transformer block 的所有构造参数外，至少还需要：

- `vocab_size`: int: 词表大小，用于确定 token embedding matrix 的维度；
- `context_length`: int: 最大上下文长度，用于确定 RoPE sin/cos buffer 大小；
- `num_layers`: int: Transformer block 数量。

实现 `adapters.run_transformer_lm` 后运行：

```
uv run pytest -k test_transformer_lm
```

提交内容 (Deliverable): 一个通过测试的 Transformer LM 模块。

资源核算 (Resource accounting)

理解 Transformer 各部分消耗多少计算与内存非常重要。这里先进行基础的 FLOPs accounting。Transformer 中绝大多数 FLOPs 来自矩阵乘法，因此核心方法只有两步：

1. 列出一组 Transformer forward pass 中的全部矩阵乘法；
2. 把每次矩阵乘法换算成 FLOPs。

有一个基本规则：若 $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$ ，则矩阵乘积 AB 需要

$$2mnp \text{ FLOPs.}$$

原因是 $(AB)[i, j] = A[i, :] \cdot B[:, j]$ ，每个点积需要 n 次乘法与 n 次加法，即 $2n$ FLOPs；一共有 mp 个输出元素，所以总 FLOPs 为 $(2n)(mp) = 2mnp$ 。

做下一题之前，建议逐一检查 Transformer block 与完整 Transformer LM 中的每个组件，把所有矩阵乘法及其 FLOPs 都列出来。

Problem (transformer_accounting): Transformer LM 资源核算 (5 分)

(a) 考虑一个采用本作业架构、规模类似 GPT-2 XL 的模型：

- `vocab_size` = 50,257
- `context_length` = 1,024
- `num_layers` = 48
- `d_model` = 1,600
- `num_heads` = 25
- `d_ff` = 4,288 ($\frac{8}{3} \times 1600$ 附近最近的 64 倍数)

这个模型有多少可训练参数？若每个参数用单精度浮点数表示，仅加载模型本身需要多少内存？

提交内容 (Deliverable): 用 1-2 句话回答。

- (b) 列出完成一次上述 GPT-2 XL-shaped 模型 forward pass 所需的矩阵乘法。假设输入序列长度等于 `context_length`，这些矩阵乘法总共需要多少 FLOPs?

提交内容 (Deliverable): 列出矩阵乘法 (说明用途) 并给出总 FLOPs。

- (c) 根据上面的分析，模型中哪些部分消耗最多 FLOPs?

提交内容 (Deliverable): 用 1-2 句话回答。

- (d) 对 GPT-2 small (12 层, $d_{\text{model}} = 768$, 12 heads)、GPT-2 medium (24 层, 1024, 16 heads)、GPT-2 large (36 层, 1280, 20 heads) 重复分析。模型规模增大时，Transformer LM 各组件占总 FLOPs 的比例如何变化?

提交内容 (Deliverable): 对每个模型给出各组件 FLOPs 及其占 forward 总 FLOPs 的比例，并用 1-2 句话描述趋势。

- (e) 取 GPT-2 XL，把 context length 增加到 16,384。一次 forward pass 的总 FLOPs 如何变化? 各组件相对 FLOPs 占比如何变化?

提交内容 (Deliverable): 用 1-2 句话回答。

4 训练 Transformer 语言模型

现在，我们已经有了数据预处理流程 (tokenizer) 和模型 (Transformer)。剩下的工作是搭建训练所需的全部基础设施，主要包括：

- **损失函数 (Loss):** 定义交叉熵 (cross-entropy) 损失；
- **优化器 (Optimizer):** 定义用于最小化该损失的 AdamW；
- **训练循环 (Training loop):** 构建加载数据、保存检查点 (checkpoint) 以及管理训练过程所需的基础设施。

4.1 交叉熵损失

回顾一下：对于任意长度为 $m+1$ 的序列 x ，Transformer 语言模型会对每个 $i = 1, \dots, m$ 定义条件分布 $p_{\theta}(x_{i+1} | x_{1:i})$ 。给定由长度为 $m+1$ 的序列组成的训练集 D ，标准交叉熵 (cross-entropy，也即 negative log-likelihood，负对数似然) 损失定义为

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^m -\log p_{\theta}(x_{i+1} | x_{1:i}). \quad (16)$$

注意，一次 Transformer forward pass 就能同时得到所有 $i = 1, \dots, m$ 对应的 $p_{\theta}(x_{i+1} | x_{1:i})$ 。

更具体地，Transformer 在每个位置 i 计算一个 logits 向量 $o_i \in \mathbb{R}^{\text{vocab_size}}$ ，于是

$$p(x_{i+1} | x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \quad (17)$$

这里 $o_i[k]$ 表示向量 o_i 在索引 k 处的值。通常，我们把交叉熵损失视为相对于 logits 向量 o_i 与目标 token x_{i+1} 定义；等价地，它是目标 x_{i+1} 上的 Dirac delta 分布与预测分布 $\text{softmax}(o_i)$ 之间的交叉熵。

实现 cross-entropy 时，和实现 softmax 一样，需要特别处理数值稳定性问题。

Problem (cross_entropy): 实现交叉熵 (1 分)

提交内容 (Deliverable): 编写函数计算交叉熵损失。输入为预测 logits o_i 与目标 x_{i+1} , 计算

$$\ell_i = -\log \text{softmax}(o_i)[x_{i+1}].$$

实现时应满足:

- 为提高数值稳定性, 先减去最大元素;
- 在可能的地方消去相互抵消的 $\log$ 与 $\exp$;
- 支持任意附加的 batch-like 维度, 并对 batch 求平均。与第 3.2 节相同, 假设所有 batch-like 维度都位于最前面, 词表维度位于最后。

实现 `adapters.run_cross_entropy`, 然后运行:

```
uv run pytest -k test_cross_entropy
```

困惑度 (Perplexity) 交叉熵足以用于训练, 但评估模型时通常还会报告 perplexity。对于一个长度为 m 、逐 token 交叉熵损失为 $\ell_1, \dots, \ell_m$ 的序列, 定义

$$\text{perplexity} = \exp \left(\frac{1}{m} \sum_{i=1}^m \ell_i \right). \quad (18)$$

4.2 SGD 优化器

有了损失函数之后, 接下来开始研究优化器。最简单的基于梯度的优化器是随机梯度下降 (Stochastic Gradient Descent, SGD)。从随机初始化参数 θ_0 开始, 对每个训练步 $t = 0, \dots, T-1$ 做更新:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

其中, B_t 是从数据集 D 中随机抽取的 batch, 学习率 α_t 与 batch size $|B_t|$ 都是超参数。

4.2.1 在 PyTorch 中实现 SGD

为了实现自己的 optimizer, 本作业要求继承 PyTorch 的 `torch.optim.Optimizer`。一个 Optimizer 子类必须实现两个方法:

`__init__(self, params, ...)` 初始化 optimizer。这里的 `params` 是待优化参数的集合; 也可以是 parameter groups, 以便对模型不同部分使用不同超参数 (例如不同 learning rate)。务必把 `params` 传给基类构造函数, 基类会保存这些参数, 之后 `step` 会使用它们。Optimizer 还可以接收额外参数 (例如 learning rate), 并把默认超参数作为字典传入基类构造函数, 字典键名由实现者自行定义。

`step(self)` 执行一次参数更新。在训练循环中, 它会在 backward pass 之后被调用, 因此此时可以访问上一个 batch 产生的梯度。该方法应遍历每个参数张量 `p`, 并原地修改 `p.data`。如果梯度存在, `p.grad` 保存的就是损失相对于该参数的梯度张量。

PyTorch optimizer API 有一些细节, 下面通过一个例子说明。为了让例子更有内容, 我们实现

一个 SGD 变体：从初始 learning rate α 开始，并随训练逐步衰减学习率：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t). \quad (20)$$

对应的 PyTorch Optimizer 写法如下（仅翻译注释，接口与代码结构保持原文）：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # 取得学习率
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p] # 取得与 p 关联的状态
                t = state.get("t", 0) # 取得迭代编号；不存在则为 0
                grad = p.grad.data # 损失相对于 p 的梯度
                p.data -= lr / math.sqrt(t + 1) * grad # 原地更新权重
                state["t"] = t + 1 # 迭代编号加一

        return loss
```

在 `__init__` 中，把 parameters 与默认 hyperparameters 一起传给基类构造函数。参数可能已被组织成多个 group，并且每组拥有不同超参数；如果只是一个 `torch.nn.Parameter` 集合，基类会创建单一 group 并赋予默认超参数。

在 `step` 中，先遍历 parameter group，再遍历该组的参数，应用式 (20)。这里迭代次数是每个 parameter 自己关联的一项 state：读取 t ，用于当前 gradient update，然后更新它。API 还允许用户传入 callable `closure`，在 optimizer step 前重新计算 loss。本作业不会依赖这一特性，但为了符合 API，示例仍保留它。

下面是一个最小训练循环：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # 清空所有可学习参数的梯度
    loss = (weights**2).mean() # 计算标量损失
    print(loss.cpu().item())
    loss.backward() # 反向传播，计算梯度
```

```
opt.step() # optimizer 更新一步
```

这就是典型 training loop 的基本结构：每次 iteration 计算 loss，执行 backward，然后执行 optimizer step。训练语言模型时，可学习参数来自 model (PyTorch 中 `m.parameters()` 会给出这一集合)，loss 则由采样得到的 batch 计算，但整体结构不变。

Problem (learning_rate_tuning): 调节学习率 (1 分)

学习率是对训练影响最大的超参数之一。用上面的 SGD toy example，把 learning rate 分别改成 `1e1`、`1e2`、`1e3`，每种只训练 10 次 iteration。观察每种 learning rate 下 loss 的行为：下降更快、下降更慢，还是发散（即训练过程中持续增大）？

提交内容 (Deliverable): 用 1-2 句话描述观察到的行为。

4.3 AdamW

现代语言模型通常不使用普通 SGD，而使用更复杂的优化器。近期常见 optimizer 大多源自 Adam [D. P. Kingma et al., 2015]。本作业使用 AdamW [I. Loshchilov et al., 2019]。AdamW 对 Adam 做了一个重要修改：加入**权重衰减 (weight decay)**，即每次迭代把参数向 0 拉近，而且该操作与 gradient update 解耦 (decoupled)。本作业按照 Loshchilov 等人 [23] 的 Algorithm 2 实现 AdamW。

AdamW 是**有状态 (stateful)** 的：对于每个 parameter，它都会维护一阶矩与二阶矩的运行估计。因此，AdamW 用额外内存换取更好的稳定性和收敛表现。

除了 learning rate α ，AdamW 还有 (β_1, β_2) 两个超参数控制 moment estimates 的更新，以及 weight decay rate λ 。常见设置是 (0.9, 0.999)；而 LLaMA 与 GPT-3 等大模型常使用 (0.9, 0.95)。另有一个很小的 ε (例如 10^{-8})，用于在 v 极小时保证数值稳定。

Algorithm 1: AdamW 优化器伪代码

init (θ)	▷ 初始化可学习参数
$m \leftarrow 0$	▷ 一阶矩向量初值，与 θ 同形
$v \leftarrow 0$	▷ 二阶矩向量初值，与 θ 同形
for $t = 1, \dots, T$:	
采样数据 batch B_t	
$g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$	▷ 计算 loss 梯度
$\alpha_t \leftarrow \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}$	▷ 计算第 t 次迭代的校正学习率
$\theta \leftarrow \theta - \alpha \lambda \theta$	▷ 应用 weight decay
$m \leftarrow \beta_1 m + (1 - \beta_1) g$	▷ 更新一阶矩估计
$v \leftarrow \beta_2 v + (1 - \beta_2) g^2$	▷ 更新二阶矩估计
$\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v} + \varepsilon}$	▷ 应用经 moment 调整的参数更新
end for	

注意：这里 t 从 1 开始。

Problem (adamw): 实现 AdamW (2 分)

提交内容 (Deliverable): 将 AdamW 实现为 `torch.optim.Optimizer` 的子类。 `__init__` 应接收 learning rate α , 以及 β 、 ε 和 λ 超参数。基类 `Optimizer` 提供 `self.state` 字典, 它把 `nn.Parameter` 对象映射到另一个用于存放该参数状态的字典; AdamW 的 moment estimates 就应保存在其中。

实现 `adapters.get_adamw_cls` 并运行:

```
uv run pytest -k test_adamw
```

Problem (adamw_accounting): AdamW 训练资源核算 (2 分)

计算运行 AdamW 所需的内存和计算量。假设每个 tensor 都使用 `float32`。

- (a) AdamW 的 peak memory 是多少? 请把结果分解成 parameters、activations、gradients、optimizer state 四部分, 并用 `batch_size` 与模型超参数 `vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads` 表示。假设

$$d_{\text{ff}} = \frac{8}{3}d_{\text{model}}.$$

为简化 activation memory 核算, 只考虑以下组件:

- Transformer block:
 - RMSNorm;
 - multi-head self-attention 子层: QKV projections、 QK^T 矩阵乘法、softmax、values 的加权和、output projection;
 - position-wise feed-forward (SwiGLU): W_1 、 W_2 、gate branch 上的 SiLU、element-wise product、 W_3 ;
- final RMSNorm;
- output embedding;
- logits 上的 cross-entropy。

提交内容 (Deliverable): 分别给出 parameters、activations、gradients、optimizer state 的代数表达式, 并给出总和。

- (b) 把上面的结果具体代入 GPT-2 XL-shaped 模型, 得到只依赖 `batch_size` 的表达式。在 80 GB memory 内, 最大 batch size 是多少?

提交内容 (Deliverable): 给出形如 $a \cdot \text{batch_size} + b$ 的数值表达式, 以及最大 batch size。

- (c) 执行一次 AdamW step 需要多少 FLOPs?

提交内容 (Deliverable): 给出代数表达式并简要说明。

- (d) **模型 FLOPs 利用率 (Model FLOPs Utilization, MFU)** 定义为: 实际观测吞吐 (tokens/s) 相对于硬件理论 peak FLOP throughput 的比例 [A. Chowdhery et al., 2022]。NVIDIA H100 对 “float32” (实际是 TensorFloat-32, 原文称其本质上为 “bfloat19”) 操作的理论峰值为 495 teraFLOP/s。若能达到 50% MFU, 在单张 H100 上训练 GPT-2 XL, 训练 400K steps、batch size 1024, 需要多长时间? 按照 Kaplan 等人 [25] 与 Hoffmann 等人 [26], 假设 backward pass 的 FLOPs 是 forward pass 的两倍。

提交内容 (Deliverable): 给出训练所需小时数, 并作简要说明。

4.4 学习率调度

使 loss 下降最快的 learning rate 往往会随训练阶段变化。训练 Transformer 时，通常使用**学习率调度 (learning rate schedule)**：训练开始使用较大的 learning rate，以便快速更新；随后随着训练进行逐渐衰减到较小值。⁶ 本作业实现训练 LLaMA 时使用的**余弦退火 (cosine annealing)** schedule。

Scheduler 本质上只是一个函数：输入当前 step t 与相关参数（例如初始和最终 learning rate），输出 step t 应用于 gradient update 的 learning rate。最简单的 schedule 是常数函数——无论输入 t 是多少，都返回同一 learning rate。

这里的 cosine annealing schedule 接收：当前 iteration t 、最大 learning rate $\alpha_{\max}$ 、最小（最终）learning rate $\alpha_{\min}$ 、warm-up iterations 数 T_w 、以及 cosine annealing 的最终 iteration T_c 。定义为：

预热 (Warm-up) 若 $t < T_w$,

$$\alpha_t = \frac{t}{T_w} \alpha_{\max}.$$

余弦退火 (Cosine annealing) 若 $T_w \leq t \leq T_c$,

$$\alpha_t = \alpha_{\min} + \frac{1}{2} \left(1 + \cos \left(\frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min}).$$

退火之后 (Post-annealing) 若 $t > T_c$,

$$\alpha_t = \alpha_{\min}.$$

Problem (learning_rate_schedule): 实现带 warmup 的 cosine learning rate schedule (1 分)

编写函数，输入 t 、 $\alpha_{\max}$ 、 $\alpha_{\min}$ 、 T_w 、 T_c ，按照上面的 schedule 返回 learning rate α_t 。然后实现 `adapters.get_lr_cosine_schedule` 并运行：

```
uv run pytest -k test_get_lr_cosine_schedule
```

4.5 梯度裁剪

训练过程中，有些样本可能产生非常大的 gradient，从而使训练不稳定。实践中常用**梯度裁剪 (gradient clipping)** 缓解这一问题：每次 backward pass 之后、optimizer step 之前，对 gradient norm 设置上限。

把所有参数的梯度统一记为 g ，计算其 ℓ_2 范数 $\|g\|_2$ 。如果该范数小于最大值 M ，保持 g 不变；否则把 g 乘上缩放因子

$$\frac{M}{\|g\|_2 + \varepsilon},$$

其中用很小的 ε （例如 10^{-6} ）提高数值稳定性。缩放后得到的 norm 会略小于 M 。

⁶有时也会采用让 learning rate 再次上升的 schedule (restart)，以帮助跳出局部极小值。

Problem (gradient_clipping): 实现梯度裁剪 (1 分)

编写 gradient clipping 函数。输入为 parameter 列表与最大 ℓ_2 norm, 函数应原地修改各 parameter 的 gradient。使用 $\varepsilon = 10^{-6}$ (PyTorch 默认值)。实现 `adapter adapters.run_gradient_clipping` 并运行:

```
uv run pytest -k test_gradient_clipping
```

5 训练循环

终于可以把前面构建的主要组件——tokenized data、model、optimizer——组合起来。

5.1 数据加载器

例如在 `tokenizer_experiments` 中得到的 tokenized data, 是单一 token 序列

$$x = (x_1, \dots, x_n).$$

即使原始数据由多个独立文档组成 (网页、源代码文件等), 常见做法仍是把所有文档拼成一条连续 token 序列, 并在文档之间加入 delimiter (例如 `<|endoftext|>`)。

Data loader 把这条序列变成一个 batch stream。每个 batch 由 B 条长度为 m 的输入序列, 以及与其对应、同样长度为 m 的 next-token targets 组成。例如当 $B = 1, m = 3$ 时,

$$([x_2, x_3, x_4], [x_3, x_4, x_5])$$

就是一个可能的 batch。

这样的 data loading 方式有几个优点。第一, 任意 $1 \leq i \leq n - m$ 都能定义合法的训练序列, 因此采样极其直接。第二, 所有训练序列长度相同, 不需要 padding, 从而提高硬件利用率, 也方便增大 batch size B 。第三, 采样时不需要把整个 dataset 全部读入内存, 因此可以处理大到无法一次装入内存的数据集。

Problem (data_loading): 实现数据加载 (2 分)

提交内容 (Deliverable): 编写函数, 输入:

- 一个 NumPy array x (保存 token IDs 的整数数组);
- `batch_size`;
- `context_length`;
- PyTorch device 字符串 (如 `'cpu'` 或 `'cuda:0'`)。

返回一对 tensor: 采样得到的输入序列与相应 next-token targets。二者 shape 都应为 `(batch_size, context_length)`, 内容为 token IDs, 并且都放到用户指定 device 上。

实现 `adapter adapters.run_get_batch`, 然后运行:

```
uv run pytest -k test_get_batch
```


低资源提示：在 CPU 或 Apple Silicon 上加载数据

如果准备在 CPU 或 Apple Silicon 上训练 LM，需要把数据移动到正确 device；模型稍后也应使用同一 device。

- CPU：使用 device string 'cpu'；
- Apple Silicon (M 系列芯片)：使用 'mps'。

关于 MPS，可参考原文列出的资源：

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

如果 dataset 大到无法全部放入内存，可以使用 Unix 系统调用 **mmap (memory mapping, 内存映射)**。它把磁盘文件映射到虚拟内存，只有在访问某段虚拟内存时才按需加载对应文件内容，于是程序可以“假装”整个 dataset 都已经在内存中。

NumPy 通过 `np.memmap` 支持该机制；如果数组最初用 `np.save` 保存，也可以用 `np.load(..., mmap_mode='r')`。二者都会返回类似 NumPy array 的对象，并在访问元素时按需加载。训练期间从 dataset 抽样时，应以 memory-mapped mode 加载数据。还必须确保指定的 `dtype` 与磁盘上数组的实际 `dtype` 一致，并建议显式检查映射后的数据是否合理，例如是否出现超出预期 vocabulary size 的值。

5.2 检查点 (Checkpointing)

除了加载数据，训练过程中还需要保存模型。真实训练任务经常需要在中途停止后继续运行，例如 job timeout 或机器故障；即使训练顺利结束，也经常希望之后访问中间模型，用来分析 training dynamics，或从不同训练阶段的模型采样。

一个 checkpoint 应包含**恢复训练所需的全部状态**。至少要保存 model weights；如果使用 AdamW 这种 stateful optimizer，也必须保存 optimizer state (例如 moment estimates)；为了继续 learning-rate schedule，还需要知道停止时的 iteration number。

PyTorch 为这些操作提供了统一接口：每个 `nn.Module` 都有 `state_dict()`，返回包含所有可学习 weights 的字典，并可通过 `load_state_dict()` 恢复；`torch.optim.Optimizer` 也有相同机制。最后，`torch.save(obj, dest)` 可以把对象写入文件路径或 file-like object；之后用 `torch.load(src)` 恢复。

Problem (checkpointing)：实现模型检查点 (1 分)

实现以下两个函数：

save_checkpoint(model, optimizer, iteration, out) 把 model、optimizer、iteration 的全部状态写入 file-like object out。可通过 model 与 optimizer 的 `state_dict()` 获取状态，再用 `torch.save(obj, out)` 保存。obj 常用字典，但只要后续能正确加载，也可以使用其他格式。
参数类型：

- model: `torch.nn.Module`
- optimizer: `torch.optim.Optimizer`
- iteration: `int`

- `out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

`load_checkpoint(src, model, optimizer)` 从 `src` (路径或 file-like object) 读取 checkpoint, 恢复 model 与 optimizer state, 并返回 checkpoint 中保存的 iteration number。可用 `torch.load(src)` 读取, 再调用 `model / optimizer` 的 `load_state_dict()`。

参数类型:

- `src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`
- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`

实现 `adapters.run_save_checkpoint` 与 `adapters.run_load_checkpoint`, 运行:

```
uv run pytest -k test_checkpointing
```

5.3 训练循环

现在可以把前面实现的所有组件组合成主训练脚本。由于后续会反复运行不同超参数的实验, 因此最好从一开始就让训练脚本方便配置, 例如把超参数做成 command-line arguments。

Problem (training_together): 组合所有组件 (4 分)

提交内容 (Deliverable): 编写一个脚本, 在用户提供的输入数据上运行 training loop。至少建议支持:

- 配置和控制各种 model hyperparameters 与 optimizer hyperparameters;
- 用 `np.memmap` 以内存高效方式加载大型 training / validation datasets;
- 将 checkpoints 序列化到用户指定路径;
- 周期性记录 training 与 validation performance (例如输出到 console, 或写入 Weights & Biases 等外部服务)。

原文给出的外部服务地址为 <https://wandb.ai>。

6 生成文本

现在我们已经能够训练模型, 最后还缺一个环节: 从训练好的模型中生成文本。

回顾语言模型的输出。语言模型接收一个长度为 `sequence_length` 的整数 token 序列 (也可能带 batch 维度), 并输出形状为 `(sequence_length, vocab_size)` 的矩阵; 序列中每个位置都对应一个预测“该位置之后的下一个 token”的分布。下面将把这个输出转化为能够不断产生新 token 的采样过程。

Softmax 按照标准约定, 语言模型直接输出最终 linear layer 的结果, 即 logits。因此在采样前, 需要使用前面式 (10) 的 softmax 把 logits 归一化成概率分布。

解码(Decoding) 为了从模型生成文本, 我们先给模型一个前缀 token 序列, 也就是**提示(prompt)**, 让模型输出词表上“下一个 token”的概率分布, 再从该分布中采样一个 vocabulary item 作为下一

个输出 token。

具体地，一次 decoding step 输入序列 $x_{1...t}$ ，并按下式采样得到 x_{t+1} ：

$$P(x_{t+1} = i \mid x_{1...t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)}, \quad (21)$$

其中

$$v = \text{TransformerLM}(x_{1...t})_t \in \mathbb{R}^{\text{vocab_size}}. \quad (22)$$

TransformerLM 输出一个 (sequence_length, vocab_size) 矩阵；因为当前要预测第 t 个位置之后的 token，所以取矩阵最后一个位置的 logits。

于是，一个基本 decoder 就可以通过反复执行这个 one-step conditional sampling 来工作：把刚刚生成的 token 追加到下一步输入，持续生成，直到产生 end-of-sequence token `<|endoftext|>`，或达到用户指定的最大生成 token 数。

解码技巧 (Decoder tricks)

本作业使用的小模型有时会生成质量很低的文本。下面两个简单技巧经常能改善采样结果。

温度缩放 (Temperature scaling) 引入 temperature 参数 τ ，把 softmax 改为

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)}. \quad (23)$$

当 $\tau \rightarrow 0$ 时， v 中最大的元素会越来越占主导，softmax 输出趋向于集中在最大元素上的 one-hot vector。

核采样 / Top- p 采样 (Nucleus sampling / Top- p sampling) 第二种技巧是 nucleus sampling，也称 top- p sampling：截断低概率 token，只在累计概率质量达到阈值 p 的高概率 token 集合中采样。

令 q 是经过（可能包含 temperature scaling 的）softmax 得到的、大小为 vocab_size 的概率分布。阈值为 p 的 nucleus sampling 定义为

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j}, & i \in V(p), \\ 0, & \text{otherwise,} \end{cases} \quad (24)$$

其中 $V(p)$ 是满足

$$\sum_{j \in V(p)} q_j \geq p$$

的最小索引集合。实现时，可以先按概率从大到小排序，然后依次选入 token，直到累计概率达到 p 。

Problem (decoding): 解码 (3 分)

提交内容 (Deliverable): 实现一个从语言模型进行 decoding 的函数。建议支持：

- 为用户提供的 prompt 生成 continuation，即输入 $x_{1...t}$ ，持续采样直到遇到 `<|endoftext|>`；
- 允许用户控制最大生成 token 数；

- 给定 temperature 时, 在采样前对 next-token distribution 使用 softmax temperature scaling;
- 给定阈值时支持 Top- p sampling [A. Holtzman et al., 2020], 即 nucleus sampling。

7 实验

现在, 把前面的所有组件组合起来, 在预训练数据集上训练小型语言模型。

7.1 如何运行实验与提交结果

理解 Transformer 各架构组件设计理由的最好方式, 是亲自修改它并运行实验。这里强调的是 hands-on experience: 真正改变模型、观察训练曲线、验证猜想。

为了做到这一点, 需要能够**快速、可重复地实验, 并保留完整记录**。本作业会用一个约 17M 参数的小模型与 TinyStories 数据集进行大量实验, 以降低迭代成本。为了保持实验一致性, 需要系统地做 component ablation 和 hyperparameter variation; 为了留下记录, 需要提交 experiment log 与每个实验对应的 learning curves。

为了能够提交 loss curves, 请周期性计算 validation loss, 并同时记录 gradient steps 与 wall-clock time。可以使用 Weights & Biases 等日志系统。

Problem (experiment_log): 实验日志 (3 分)

为 training / evaluation code 建立 experiment tracking infrastructure, 使其能够按 gradient steps 和 wall-clock time 记录实验与 loss curves。

提交内容 (Deliverable): 提交实验日志基础设施代码, 以及一个 experiment log 文档, 记录本节后续各题中尝试过的所有实验。

7.2 TinyStories

首先使用一个非常简单的数据集 TinyStories [R. Eldan et al., 2023]。它训练很快, 也足以观察很多有趣的训练行为。数据获取方式见第 1 节。

Example (tinystories_example): TinyStories 样例

从前, 有一个名叫 Ben 的小男孩。Ben 很喜欢探索周围的世界。他看到过许多令人惊叹的东西, 比如商店里陈列的漂亮花瓶。有一天, Ben 正在店里走着, 突然发现了一个非常特别的花瓶。看到它时, Ben 惊呆了! 他说: “哇, 这真是一个很棒的花瓶! 我可以买下来吗?” 店主笑着说: “当然可以。你可以把它带回家, 让所有朋友看看它有多漂亮!” 于是 Ben 把花瓶带回了家, 他为它感到非常自豪! 他把朋友们叫来, 向他们展示这个漂亮的花瓶。所有朋友都觉得花瓶非常美, 也不敢相信 Ben 竟然这么幸运。Ben 就这样在商店里找到了一个令人惊叹的花瓶!

7.2.1 超参数调节

作业先给出一些基本 hyperparameters 作为起点, 其他参数需要自行寻找有效设置。

词表大小 (Vocab size)

10000。常见词表大小为几万到几十万。可以改变这一值, 观察 vocabulary 与模型行为如何变化。

上下文长度 (Context length)

256。TinyStories 这类简单数据可能不需要很长序列；稍后的 OpenWeb-Text 可以尝试改变它，并观察每次 iteration runtime 与最终 perplexity 的变化。

d_{model} 512。比许多小型 Transformer 论文常见的 768 略小，以提高实验速度。

d_{ff} 1344。大约等于 $\frac{8}{3}d_{\text{model}}$ ，同时又是 64 的倍数，有利于 GPU 性能。

RoPE Θ 10000。

层数与 head 数 4 layers, 16 heads; 合计约 17M 个不含 embedding 的参数，是一个相当小的 Transformer。

处理 token 总数 327,680,000。即 `batch size × total step count × context length` 应大致等于该值。

其余 hyperparameters 需要通过 trial and error 找到良好默认值，包括：learning rate、learning-rate warmup、AdamW 的 $(\beta_1, \beta_2, \varepsilon)$ 和 weight decay。典型设置可参考 Kingma 等人 [22]。

7.2.2 组合全部组件

现在可以训练 BPE tokenizer、对 training dataset 做 tokenization，再把 tokenized data 送入前面写好的 training loop。原文特别提醒：如果实现正确而且高效，以上设置在单张 B200 GPU 上的运行时间应大约为 20–30 分钟。如果明显更慢，请检查 data loading、checkpointing、validation-loss 计算是否成为瓶颈，并确认模型计算已经正确 batch 化。

7.2.3 调试模型架构的技巧

原文强烈建议熟悉 IDE 内置 debugger (例如 VSCode / Zed)；与大量 `print` 调试相比，它通常更节省时间。若使用文本编辑器，可以使用 `ipdb`。一些常见实践包括：

- 开发任何 neural network architecture 时，可以先尝试过拟合单个 **minibatch**。若实现正确，应能很快把 training loss 压到接近 0；
- 在不同 model component 中设置 breakpoint，检查 intermediate tensor shape 是否符合预期；
- 监控 activation、model weight、gradient 的 norm，确认它们没有 exploding 或 vanishing。

Problem (learning_rate): 调节学习率 (2 B200 小时, 3 分)

Learning rate 是最重要的 hyperparameter 之一。基于训练好的 base model，完成：

- (a) 对多个 learning rates 做 hyperparameter sweep，报告最终 losses；若 optimizer diverges，则明确标记发散。

提交内容 (Deliverable): 给出多个 learning rates 对应的 learning curves，并解释 hyperparameter search strategy。

提交内容 (Deliverable): 训练出一个在 TinyStories 上 per-token validation loss 不高于 1.45 的模型。

- (b) 经验上常说最好的 learning rate 位于“稳定性的边缘 (edge of stability)”。研究 learning rate 开始发散的位置与最佳 learning rate 之间的关系。

提交内容 (Deliverable): 给出随 learning rate 增大的一组 learning curves，其中至少包

含一次 divergent run，并分析它与 convergence rate 的关系。

低资源提示：在 CPU 或 Apple Silicon 上训练较少步数

如果使用 `cpu` 或 `mps`，原文建议把总处理 token 数降到 40,000,000；这仍足以得到相对流畅的文本。同时，可以把 validation loss 目标从 1.45 放宽到 2.00。

参考 staff solution：M4 Max、36 GB RAM，使用

$$\text{batch size} \times \text{total step count} \times \text{context length} = 32 \times 5000 \times 256 = 40,960,000$$

tokens。CPU 约耗时 1 小时 22 分钟，MPS 约 36 分钟；step 5000 时 validation loss 为 1.80。额外提示：

- 若总训练步数为 N ，建议调整 cosine learning-rate decay，使其恰好在 step N 到达 minimum learning rate；
- 使用 MPS 时，不要像 CUDA 那样启用 TF32 kernel，也就是不要设置：

```
torch.set_float32_matmul_precision('high')
```

原文称其在 PyTorch 2.9.0 的 MPS backend 上观察到部分 kernel 会静默地产生错误，从而导致训练不稳定；

- 可以用 `torch.compile` JIT 编译模型提高速度：CPU 上使用

```
model = torch.compile(model)
```

MPS 上可以用

```
model = torch.compile(model, backend="aot_eager")
```

来一定程度优化 backward pass。原文指出截至 PyTorch 2.9.0，Inductor 尚不支持 MPS。

Batch size 也很重要：更大的 batch 可以让 GPU 进行更大的矩阵乘法，从而提高效率，但并不意味着越大一定越好。接下来通过实验观察其影响。

Problem (batch_size_experiment): 改变 batch size (1 B200 小时, 1 分)

把 batch size 从 1 一直改到 GPU memory limit，中间至少测试若干取值，包括常见的 64 与 128。

提交内容 (Deliverable)：给出不同 batch sizes 的 learning curves；如有必要，应重新优化 learning rate。

提交内容 (Deliverable)：用几句话讨论 batch size 对训练产生的影响。

有了 decoder，现在可以从模型生成文本。作为参考，训练结果至少应达到下面示例的大致水平。

Example (ts_generate_example): TinyStories 语言模型生成样例

从前，有一个漂亮的女孩叫 Lily。她很喜欢吃口香糖，尤其喜欢那颗大大的黑色口香糖。有一天，Lily 的妈妈叫她来帮忙做晚饭。Lily 兴奋极了！她很喜欢帮助妈妈。Lily 的妈妈为晚饭煮了一大锅汤。Lily 非常开心，说：“谢谢你，妈妈！我爱你。”她帮妈妈把汤倒进一个大碗里。晚饭之后，Lily 的妈妈又做了一些美味的汤。Lily 很喜欢！她说：“谢谢你，妈妈！这个汤太好喝了！”妈妈笑着说：“你喜欢就好，Lily。”她们完成了烹饪，然后继续一起做饭。故事结束。

低资源提示：在 CPU 或 Apple Silicon 上生成文本

如果使用前面只处理 40M tokens 的低资源配置，生成结果仍应有英语文本的形态，但流畅度不会达到上面的水平。原文给出的一个样例如下。这里的语法与语义错误来自模型原始生成结果，翻译时刻意保留其混乱感：

从前，有一个名叫 Sue 的小女孩。Sue 有一颗她非常喜欢的牙齿。那是“他的最好的脑袋”。有一天，Sue 出去散步，遇到了一只瓢虫！他们成了好朋友，一起在小路上玩。

“嘿，Polly！我们出去吧！”Tim 说道。Sue 看着天空，发现很难找到一种方法去“跳舞发亮”。她笑了，并同意去帮助“那个说话的东西！”

当 Sue 看着天空移动时，它究竟是什么。她……

Problem (generate): 生成文本 (1 分)

使用 decoder 与训练好的 checkpoint，报告模型生成文本。可能需要调节 temperature、top- p 等 decoder parameters 才能得到较流畅的输出。

提交内容 (Deliverable): 输出至少 256 tokens 的文本（或者生成到第一个 `<|endoftext|>` 为止），并简要评论其 fluency，同时指出至少两个会影响输出质量的因素。

7.3 消融实验与架构修改

真正理解 Transformer 的有效方法之一，是亲自修改它并观察模型行为。下面进行几个简单的 ablation 与 modification。

消融 1：层归一化

人们常说 layer normalization 对 Transformer training stability 很重要。现在把每个 Transformer block 中的 RMSNorm 全部移除，观察会发生什么。

Problem (layer_norm_ablation): 移除 RMSNorm 并训练 (0.5 B200 小时, 1 分)

移除 Transformer 中所有 RMSNorm 后重新训练。使用之前的 optimal learning rate 时会发生什么？降低 learning rate 后能否重新获得稳定性？

提交内容 (Deliverable): 给出移除 RMSNorm 后的 learning curve，以及找到的最佳 learning rate 对应 learning curve。

提交内容 (Deliverable): 用几句话评论 RMSNorm 的影响。

接着研究另一个乍看之下可能显得任意的选择：layer normalization 放置位置。Pre-norm Transformer block 定义为

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)), \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

这是相对于原始 Transformer 的少数几项已经形成广泛共识的修改之一。原始 Transformer 使用 post-norm：

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)), \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \quad (28)$$

现在把实现改回 post-norm，观察训练行为。

Problem (pre_norm_ablation): 实现 post-norm 并训练 (0.5 B200 小时, 1 分)

把 pre-norm Transformer 实现改成 post-norm, 训练并观察结果。

提交内容 (Deliverable): 给出 post-norm Transformer 的 learning curve, 并与 pre-norm 进行比较。

从这些实验中可以观察到, layer normalization 会显著影响 Transformer 行为, 甚至 normalization 的放置位置本身也非常重要。

消融 2: 位置嵌入

接下来研究 positional embeddings 对模型性能的影响。比较 base model (使用 RoPE) 与完全不加入 positional embedding 的模型 (NoPE)。已有工作指出, 仅 decoder 的 Transformer (也就是像本作业这样具有 causal mask 的模型) 理论上可能在没有显式 positional embeddings 的情况下推断 relative 或 absolute position information [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]。现在通过实验比较 NoPE 与 RoPE。

Problem (no_pos_emb): 实现 NoPE (0.5 B200 小时, 1 分)

修改使用 RoPE 的 Transformer, 实现中完全移除 positional embedding information, 并观察训练结果。

提交内容 (Deliverable): 给出比较 RoPE 与 NoPE 性能的 learning curve。

消融 3: SwiGLU 与 SiLU

接下来按照 Shazeer [20] 的思路, 研究 feed-forward network 中 gating 的重要性: 比较默认 SwiGLU FFN 与只使用 SiLU activation、没有 gated linear unit (GLU) 的 FFN:

$$\text{FFN}_{\text{SiLU}}(x) = W_2 \text{SiLU}(W_1 x). \quad (29)$$

回顾默认 SwiGLU: 内层维度约为

$$d_{\text{ff}} = \frac{8}{3} d_{\text{model}},$$

并保证 $d_{\text{ff}} \bmod 64 = 0$, 以充分利用 GPU tensor cores。作为 ablation baseline, FFN_{SiLU} 应改用

$$d_{\text{ff}} = 4d_{\text{model}},$$

以便让参数量大致匹配默认 SwiGLU FFN——SwiGLU 有 3 个 weight matrices, 而 SiLU baseline 只有 2 个。

Problem (swiglu_ablation): SwiGLU vs. SiLU (0.5 B200 小时, 1 分)

提交内容 (Deliverable): 给出比较 SwiGLU 与 SiLU feed-forward networks 的 learning curve, 并使两者 parameter count 大致匹配。

提交内容 (Deliverable): 用几句话讨论观察到的结果。

低资源提示：GPU 资源有限的在线学习者可继续在 TinyStories 上测试修改

本作业接下来会转到规模更大、噪声更高的网页数据 OpenWebText，继续做 architecture modifications，并可选参加课程 leaderboard。

由于把 LM 在 OpenWebText 上训练到较流畅需要很长时间，原文建议 GPU 资源有限的在线学习者继续在 TinyStories 上测试修改，并使用 validation loss 评价性能。

7.4 在 OpenWebText 上运行

现在转向更典型的预训练数据：通过 web crawl 构建的 OpenWebText。课程也提供了一个小型 OpenWebText 样本，形式同样是单个文本文件，获取方式见第 1 节。

下面是 OpenWebText 中的一段示例。与 TinyStories 相比，它明显更加真实、复杂而且多样。原文建议浏览 training dataset，以了解 web-scraped corpus 的真实训练文本是什么样。

Example (owt_example): OWT 样例

Baseball Prospectus 的技术总监 Harry Pavlidis 在雇用 Jonathan Judge 时冒了一个险。Pavlidis 知道，正如 Alan Schwarz 在《The Numbers Game》中写道：“美国文化中，没有哪个角落像棒球运动员的表现那样，被如此精确地计数、如此热情地量化。”只要在这里那里点几下，你就能查到 Noah Syndergaard 的快速球在飞向本垒板的过程中每分钟旋转超过 2,100 圈；也能查到 2016 年 Nelson Cruz 在达到资格线的击球手中拥有全联盟最高的平均击球初速度；还能找到无数其他细节，它们仿佛直接来自电子游戏或科幻小说。不断上涨的数据海洋，也赋予了棒球文化中一个越来越重要的角色更多力量：热衷分析的业余爱好者。

这种赋权也带来了更多审视——不仅针对测量本身，也针对测量背后的人与出版物。身处 Baseball Prospectus，Pavlidis 很清楚定量分析一旦出现不完美会招来怎样的反弹。他也知道，该网站关于接球手的指标体系需要重新设计，而完成这项工作需要一个真正有学识的头脑——一个能够处理复杂统计建模问题的人。

“他把我们都吓到了。”——Harry Pavlidis

根据 Judge 的文章，以及两人在该网站赞助的一场球场活动中的互动，Pavlidis 有一种直觉：Judge “真正懂这件事”。……

注意

这个实验中，可能需要重新调节 learning rate、batch size 等 hyperparameters。

Problem (main_experiment): 在 OWT 上实验 (2 B200 小时, 2 分)

使用与 TinyStories 相同的 model architecture 和总 training iterations，在 OpenWebText 上训练语言模型。模型表现如何？

提交内容 (Deliverable): 给出 OpenWebText 上的 learning curve，并描述它与 TinyStories 的 loss 有何不同；应如何解释这些 loss？

提交内容 (Deliverable): 给出 OpenWebText LM 生成文本，格式与 TinyStories 输出相同。其 fluency 如何？为什么即便使用相同模型和 compute budget，输出质量仍然比 TinyStories 差？

7.5 自行修改模型 + Leaderboard

做到这里，整个作业基本接近完成。最后的任务是尝试改进 Transformer architecture，并观察自己的 hyperparameters 与 architecture 在班级其他同学的实现中处于什么水平。

Leaderboard 规则

除了以下限制之外，没有其他约束：

Runtime 提交程序最多只能在一张 B200 上运行 45 分钟。如果使用 SLURM 或 Modal，建议在 submission script 中显式强制这一时限。

Data 只能使用课程提供的 OpenWebText training dataset。

除此之外，可以自由使用任何想法。

如果需要 inspiration，原文建议参考：

- 当前高水平的开源 LLM family，例如 Llama 3 [A. Grattafiori et al., 2024] 或 Qwen 2.5 [A. Yang et al., 2024]；
- NanoGPT speedrun repository: <https://github.com/KellerJordan/modded-nanogpt>。社区成员会在那里分享很多用于“小规模 LM 预训练竞速”的修改。

一个可以尝试的经典修改是**权重绑定 (weight tying)**：让 input embedding 与 output embedding 使用同一组权重。这个想法可以追溯到原始 Transformer [8, Section 3.4]，也见 PaLM [16, Section 2]。如果采用 weight tying，可能需要降低 embedding / LM head initialization 的 standard deviation。

在进行完整 45-minute run 前，应先在 OpenWebText 的小子集或 TinyStories 上测试这些修改。

原文也特别提醒：Leaderboard 上对这种小规模设定有效的修改，不一定能 generalize 到更大规模 pretraining。课程后续 scaling laws 单元会进一步研究这个问题。

Problem (leaderboard): Leaderboard (10 B200 小时, 6 分)

按照上述 leaderboard rules 训练模型，目标是在 0.75 B200-hours 内尽可能降低 language model validation loss。

提交内容 (Deliverable): 报告最终记录的 validation loss；提供对应 learning curve，且横轴必须清楚显示 wall-clock time，并且总时间小于 45 分钟；同时描述具体做了哪些修改。课程期望 leaderboard submission 至少击败 validation loss 为 5.0 的 naive baseline。

提交地址: <https://github.com/stanford-cs336/assignment1-basics-leaderboard>

参考文献

文献题名说明

为保证检索与引用准确性，作者名、论文/数据集正式英文题名、会议名称、DOI 与 URL 保留原文，不再翻译。

- [1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.
- [2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.
- [3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in Proc. of ACL, 2016.

- [4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.
- [5] P. Gage, “A new algorithm for data compression,” *C Users Journal*, vol. 12, no. 2, pp. 23–38, Feb. 1994.
- [6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.
- [7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.
- [8] A. Vaswani et al., “Attention is All you Need,” in *Proc. of NeurIPS*, 2017.
- [9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. of IWSWLT*, 2019.
- [10] R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in *Proc. of ICML*, 2020.
- [11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.
- [12] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.
- [13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Proc. of NeurIPS*, 2019.
- [14] A. Grattafiori et al., “The Llama 3 Herd of Models.” [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [15] A. Yang et al., “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [16] A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022.
- [17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.
- [18] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1702.03118>
- [19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” [Online]. Available: <https://arxiv.org/abs/1612.08083>
- [20] N. Shazeer, “GLU Variants Improve Transformer.” 2020.
- [21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.
- [22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2015.
- [23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. of ICLR*, 2019.

- [24] T. B. Brown et al., “Language Models are Few-Shot Learners,” in Proc. of NeurIPS, 2020.
- [25] J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020.
- [26] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.
- [27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in Proc. of ICLR, 2020.
- [28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4344–4353. doi: 10.18653/v1/D19-1443.
- [29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in Thirty-seventh Conference on Neural Information Processing Systems, 2023. [Online]. Available: <https://openreview.net/forum?id=Dr1l2gcjz1>